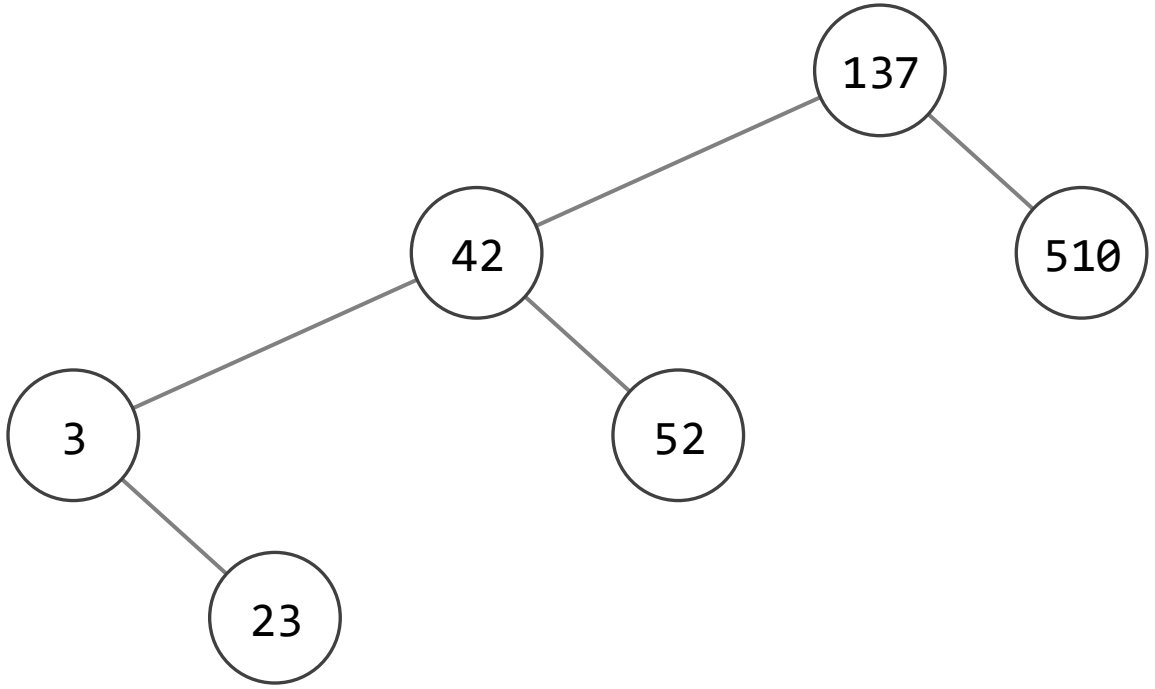
Binary Search Trees

Search and insert · Algorithms §3.2, part 1

This video

- What a **binary search tree** is (and how it's different from a heap)
- Why we can't store it in an array, the way we stored the heap
- **`get`** and **`put`**, one step at a time, counting compares
- The code: short, recursive, and a little tricky
- The **shape** of the tree depends on the order keys arrive in
- BSTs and **quicksort**: how fast is a BST, really?

What is a binary search tree?



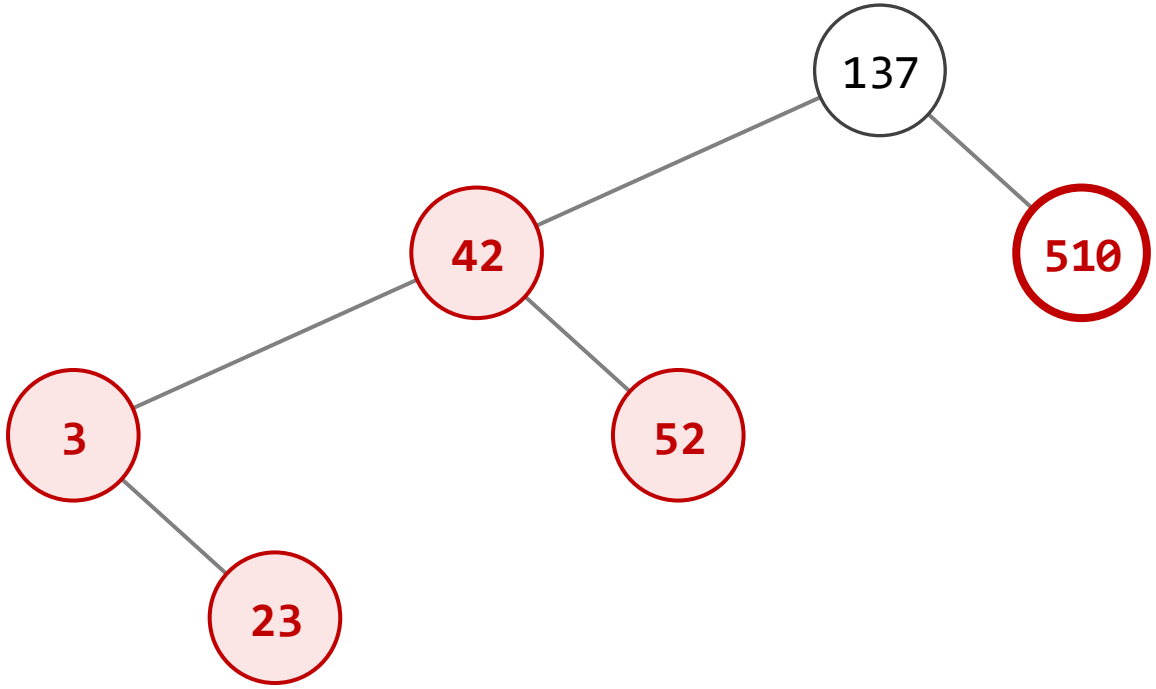
A **binary tree**: each node has at most two children, a **left** and a **right**.

In **symmetric order**: each node's key is

- **larger** than every key in its **left** subtree, and
- **smaller** than every key in its **right** subtree.

Every node holds a key and a value. The drawings only show keys.

Symmetric order



Everything in 137's **left** subtree is smaller than 137: 3, 23, 42, 52.

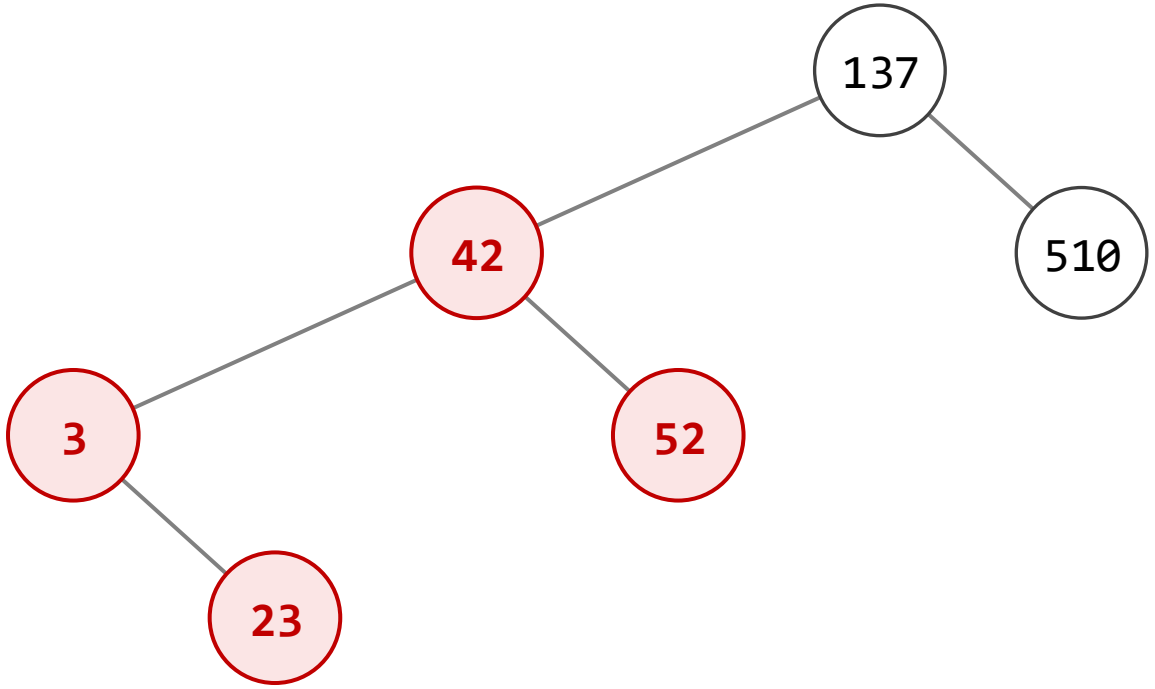
Everything in its **right** subtree is bigger: 510.

And the same is true at **every** node, not just the root. Check 42: 3 and 23 on its left, 52 on its right.

keys < 137 (left subtree)

keys > 137 (right subtree)

A recursive structure



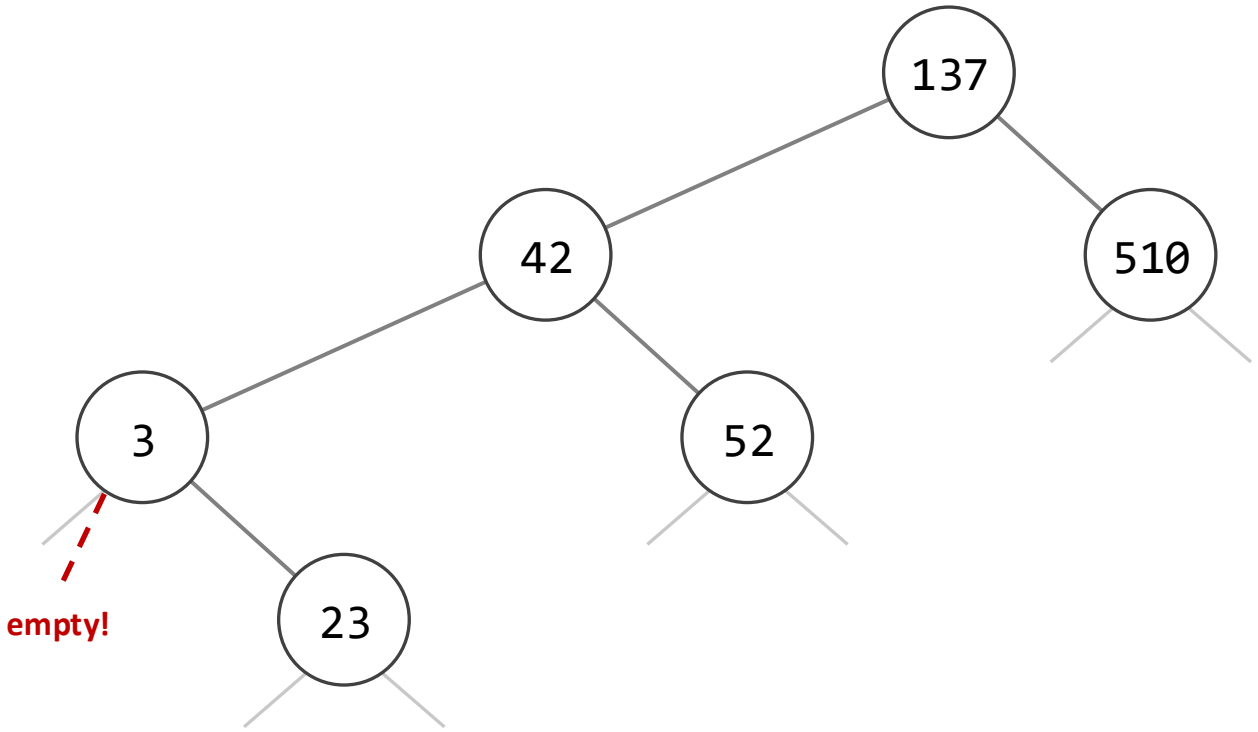
A binary tree is either

- **empty** (`null`), or
- a **node** with links to two binary trees, a left one and a right one.

So 137's left subtree (highlighted) is a whole binary search tree on its own.

When you see a definition like this, think **recursion.**

A recursive structure



3's **left** subtree is **empty**. That still counts as a binary tree.

The gray stubs are `null` links: every one of them is an empty tree.

Empty trees are the **base case** of every recursive method we'll write.

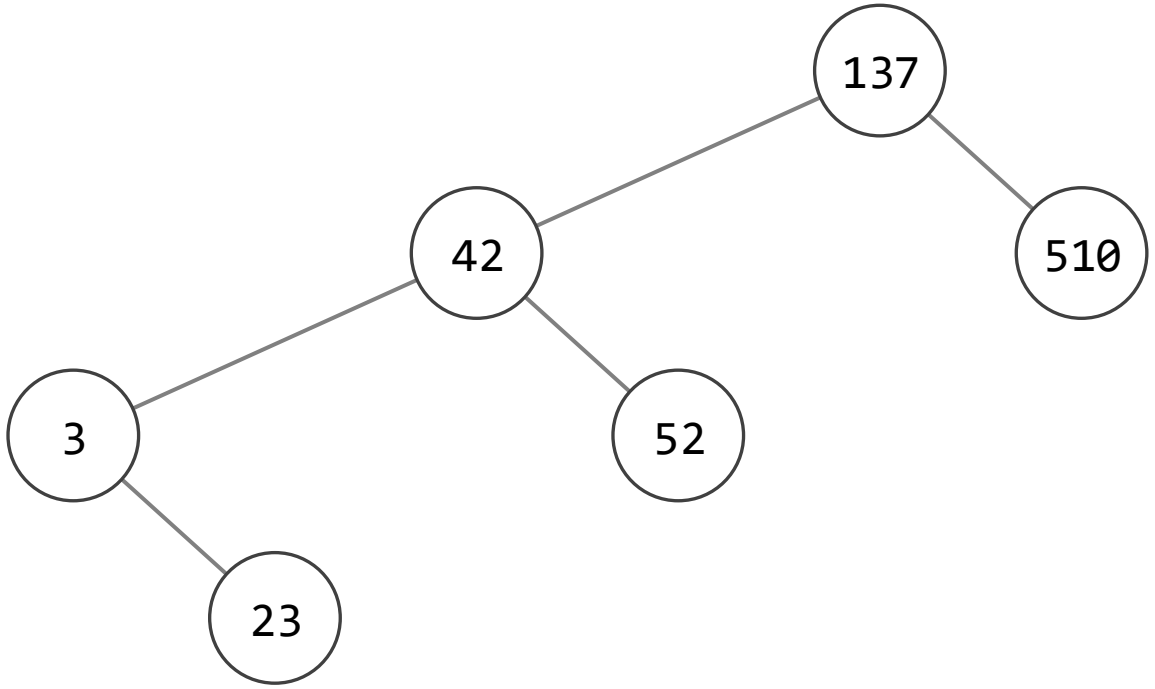
(A tree with n nodes always has n + 1 null links. Count them: 6 nodes, 7 nulls.)

BSTs vs. heaps

	binary heap	**binary search tree**
the rule	parent ≥ each child	left subtree < node < right subtree
left vs. right	no rule at all	left is always smaller
shape	complete: every level full, bottom fills from the left	any shape at all
largest key	at the root	all the way down to the right
stored in	an array	linked nodes (next slides)
good at	`insert`, `delMax`	search, and anything about order

Both are binary trees, and both are drawn upside-down. That's where the resemblance ends.

A BST in an array, like the heap?



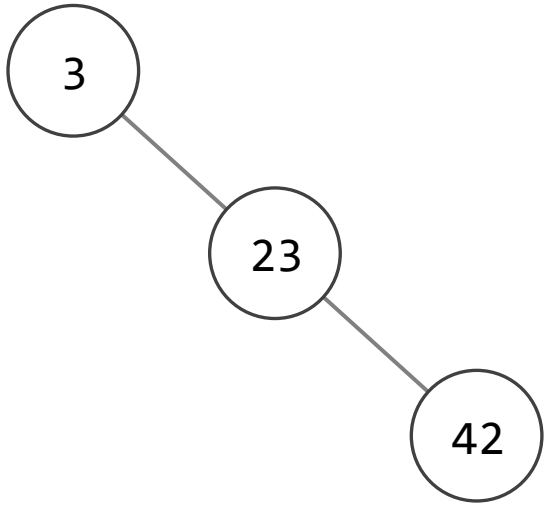
	0	1	2	3	4	5	6	7	8	9
a		137	42	510	3	52	null	null	null	23

Same numbering as the heap: children of k at **2k** and **2k + 1**.

The heap is **complete**, so there are no gaps. A BST can be any shape.

Here: 6 keys, but 23 lands at index 9, with **3 empty slots** in the middle.

It gets much worse





Insert keys in **sorted order** and every new key goes to the right: indices 1, 3, 7, 15, 31, ...

The k-th key lands at index $\mathbf{2^k - 1}$.

3 keys: 7 slots. 10 keys: 1,023. **30 keys: over a billion.**

So: no array. We'll use **linked nodes** instead.

Linked nodes

```
1  public class BST<Key extends Comparable<Key>, Value> {
2      private Node root;              // the root
3
4      private class Node {
5          private Key key;
6          private Value val;
7          private Node left, right;   // subtrees
8
9          public Node(Key key, Value val) {
10             this.key = key;
11             this.val = val;
12         }
13     }
14     ...
15 }
```

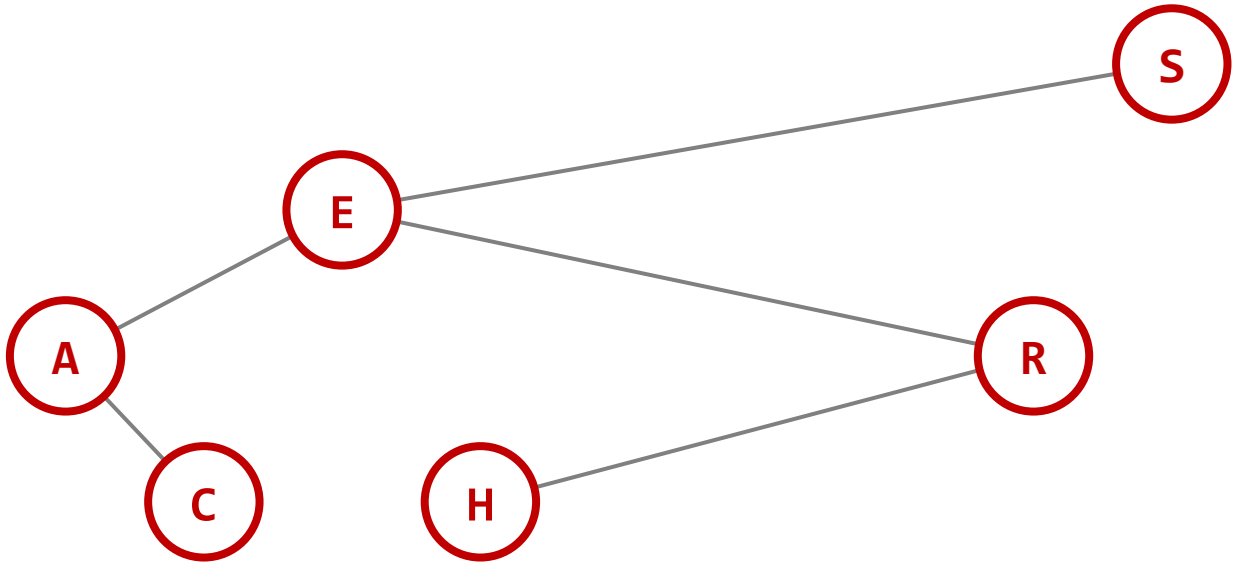
Like a linked-list node, but with **two** links.

The whole tree is just a link to the **root**.

Every step down the tree is one `compareTo()`, so keys must have one.

An empty tree is `root == null`.

Building the tree: the first six puts



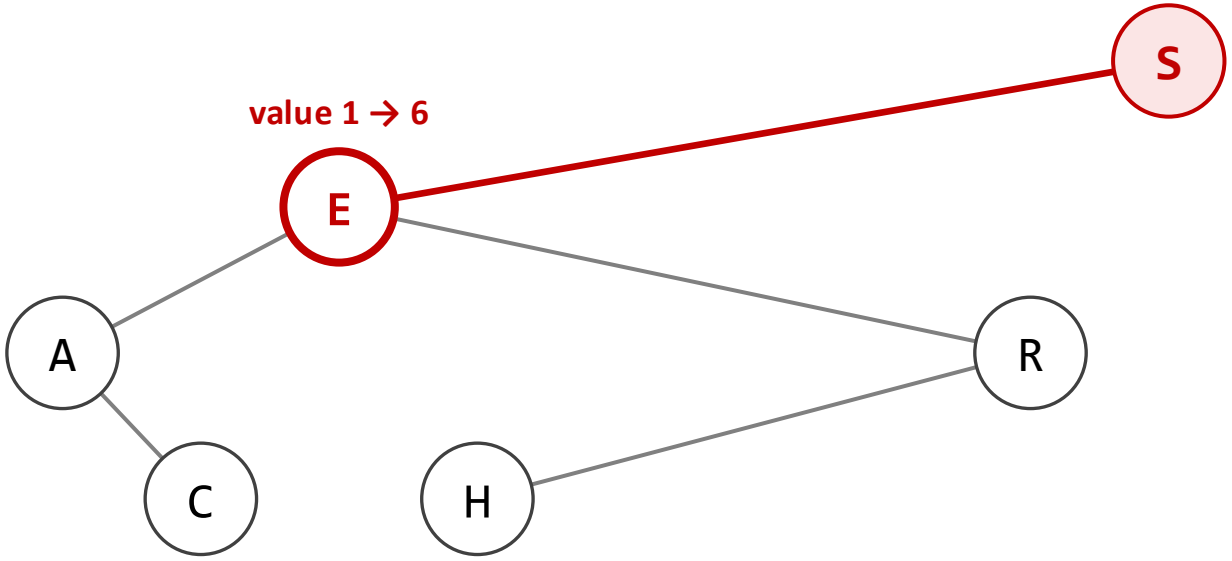
The test client from last time: put S E A R C H E X A M P L E, each with its position.

S came **first**, so it's the root, forever.

Each later key walks down from S (less: left, greater: right) and hangs a new node off the first `null` link it reaches.

new keys (6 puts, 6 keys)

Building the tree: `put(E, 6)`



E < S: left. E = E: **it's already here**.

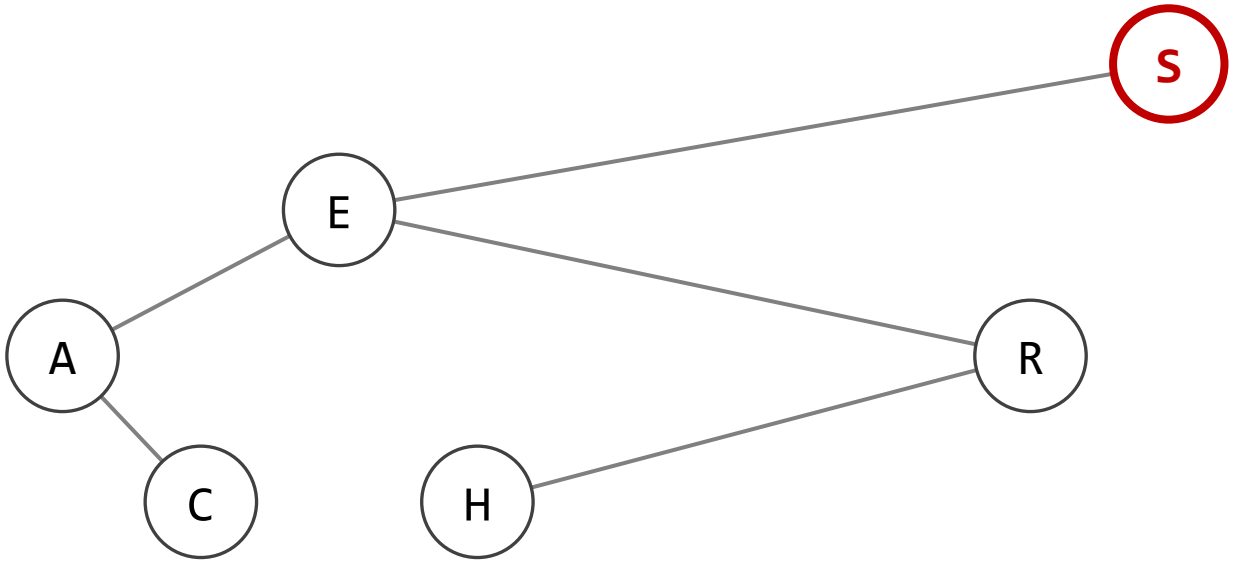
`put` doesn't add a second E. It **replaces** E's value: 1 becomes 6.

The tree doesn't change shape. Still 6 keys.

value replaced

the path

Building the tree: `put(X, 7)`



Next: X. Start at the root, as always.

X is **greater** than S. Go **right**.

cost so far

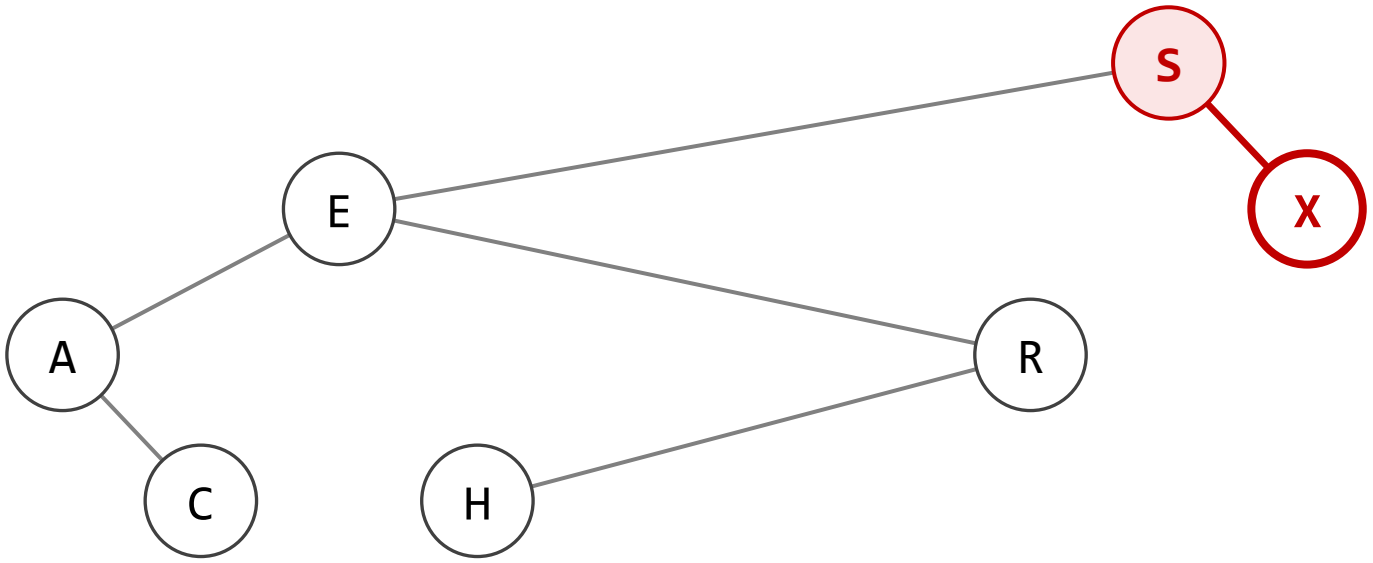
compares **1**

X vs. S

the node we're comparing with

the path so far

Building the tree: `put(X, 7)`



S's right link is `null`: X isn't in the tree.

Hang a **new node** there. 1 compare, and nothing else moves.

cost so far

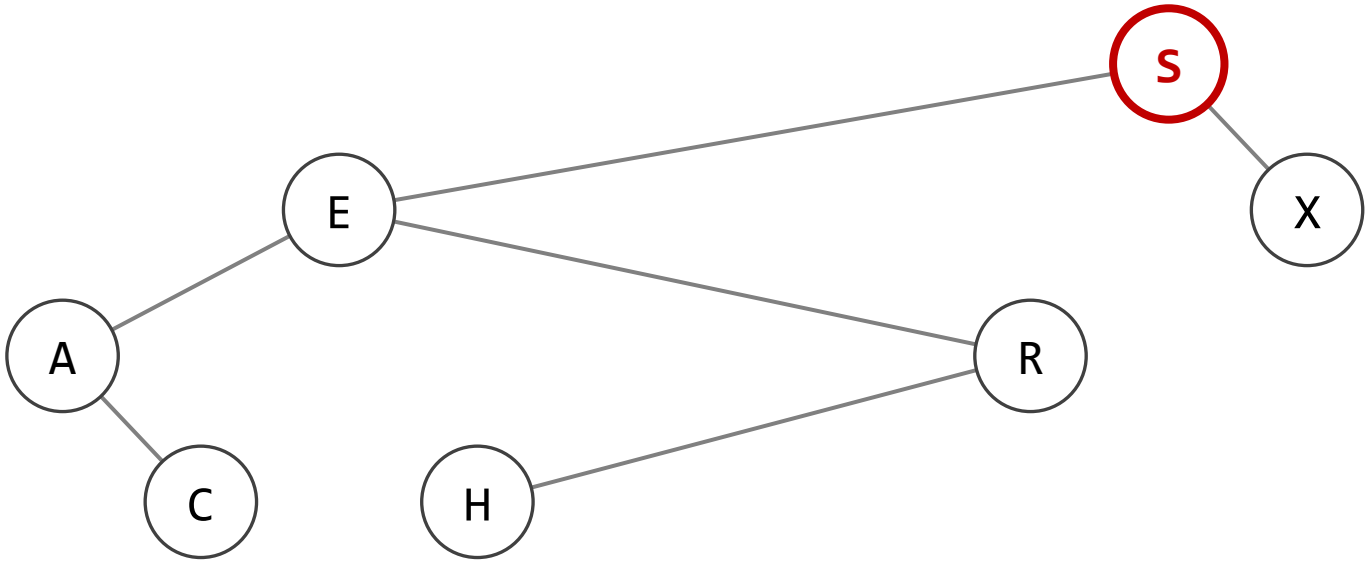
compares	1

adding X: no compares

the new node

the search path

Building the tree: `put(A, 8)`



A is **less** than S. Go **left**.

cost so far

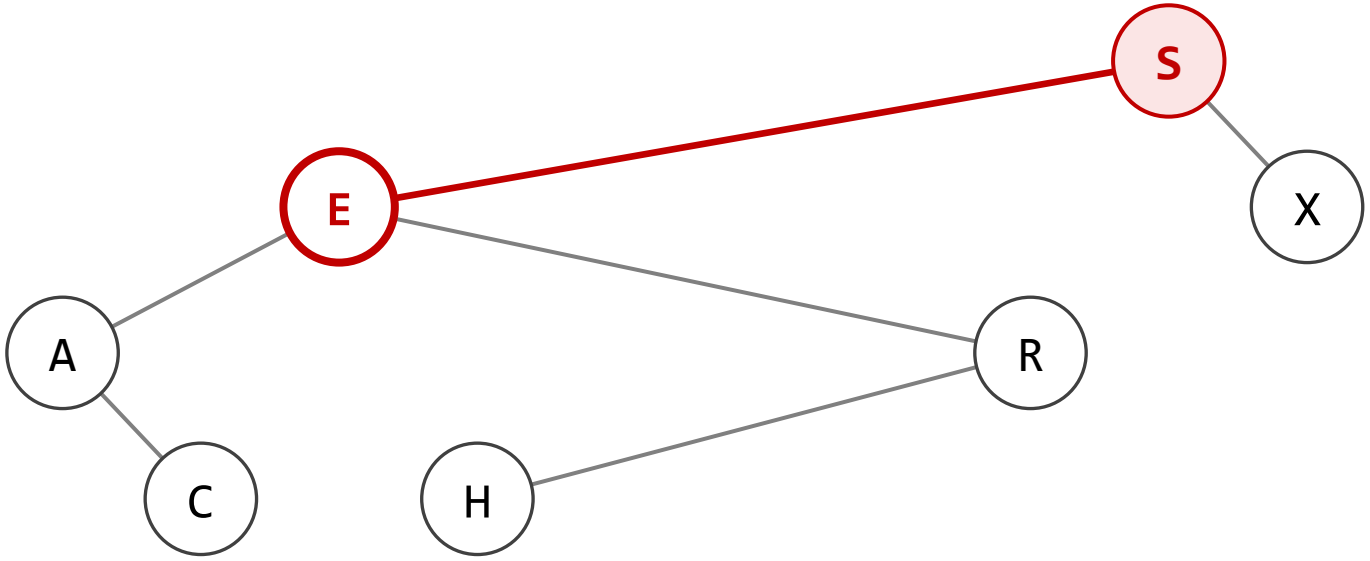
compares **2**

A vs. S

the node we're comparing with

the path so far

Building the tree: `put(A, 8)`



A is **less** than E. Go **left**.

cost so far

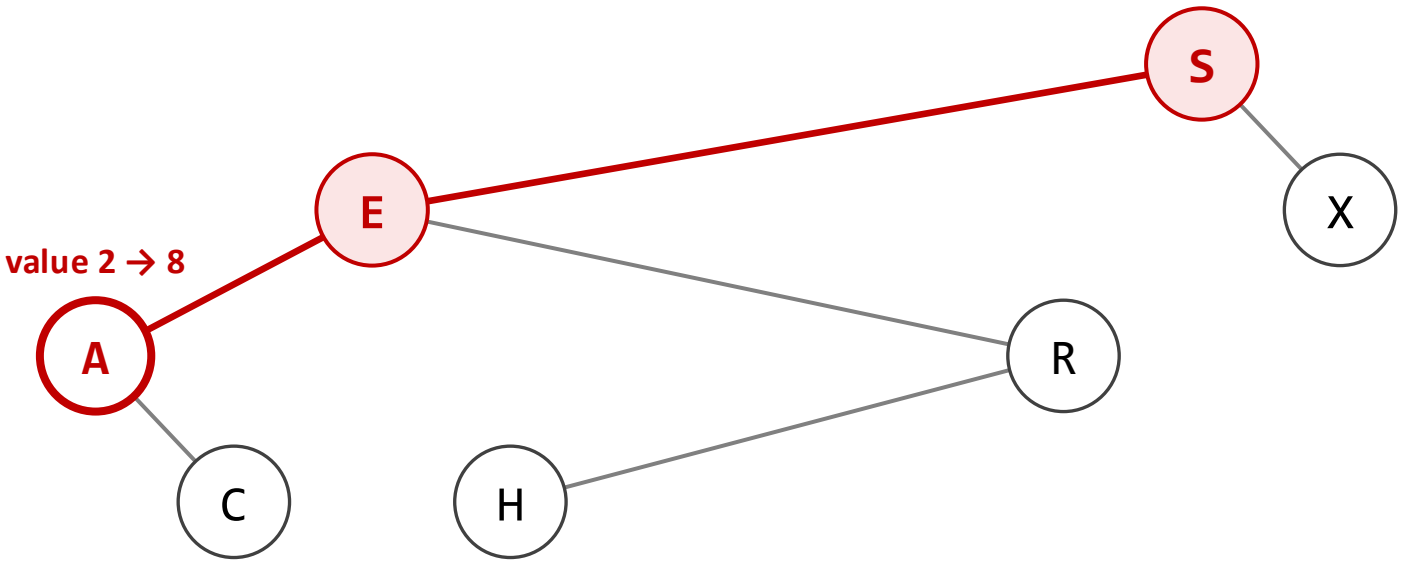
compares **3**

A vs. E

the node we're comparing with

the path so far

Building the tree: `put(A, 8)`



A **equals** A: **it's already here**.

Replace its value: 2 becomes 8. No new node; the tree doesn't change shape.

cost so far

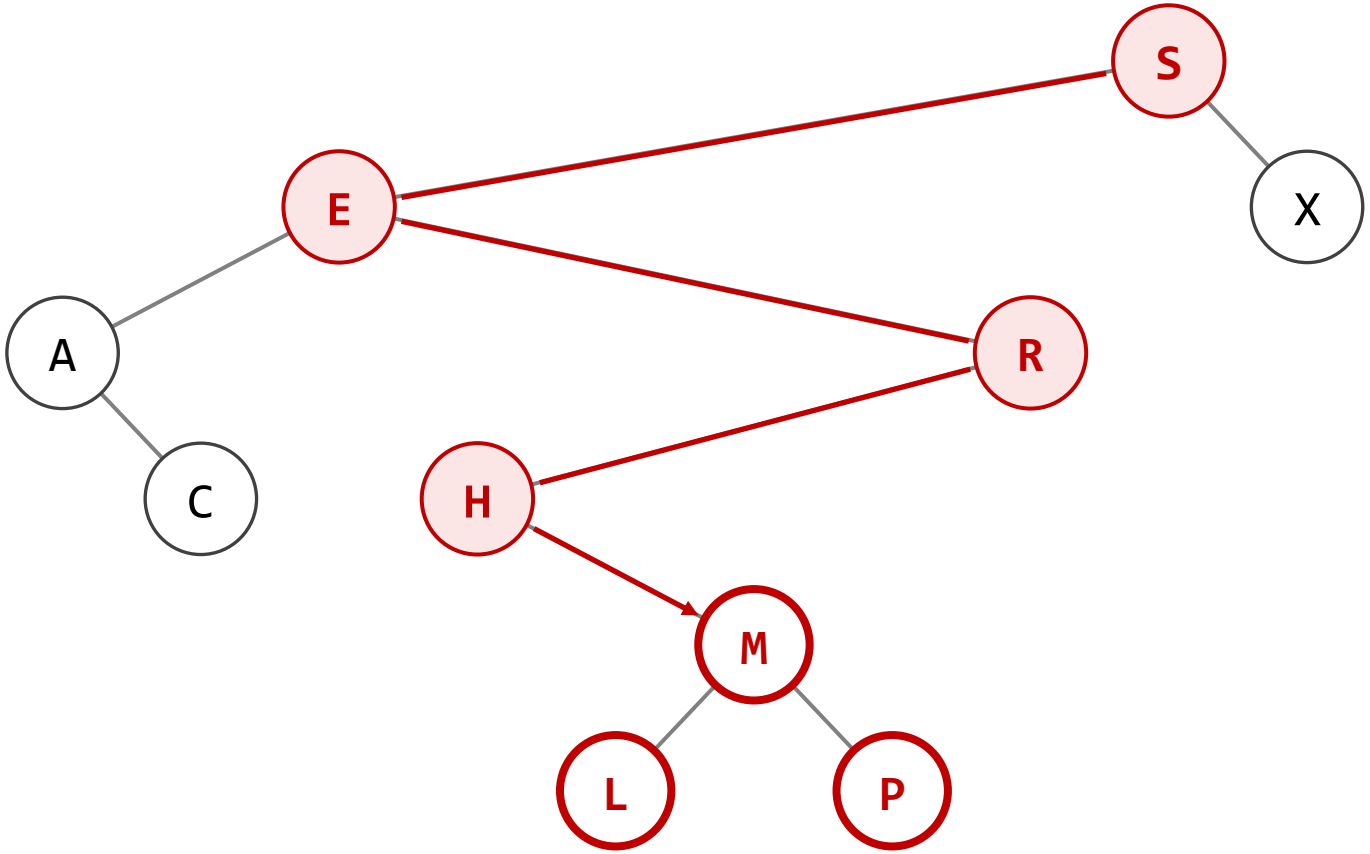
compares **4**

A vs. A: equal

the node we're comparing with

the path so far

Building the tree: `put(M)`, `put(P)`, `put(L)`



M, P, L are all new. Each walks down S, E, R, H (and then M) to a `null` link: **4, 5 and 5** compares.

L and P land 6 levels down. Every later search for them pays for that.

A million keys inserted in random order: about **28** compares per `put`.

cost so far

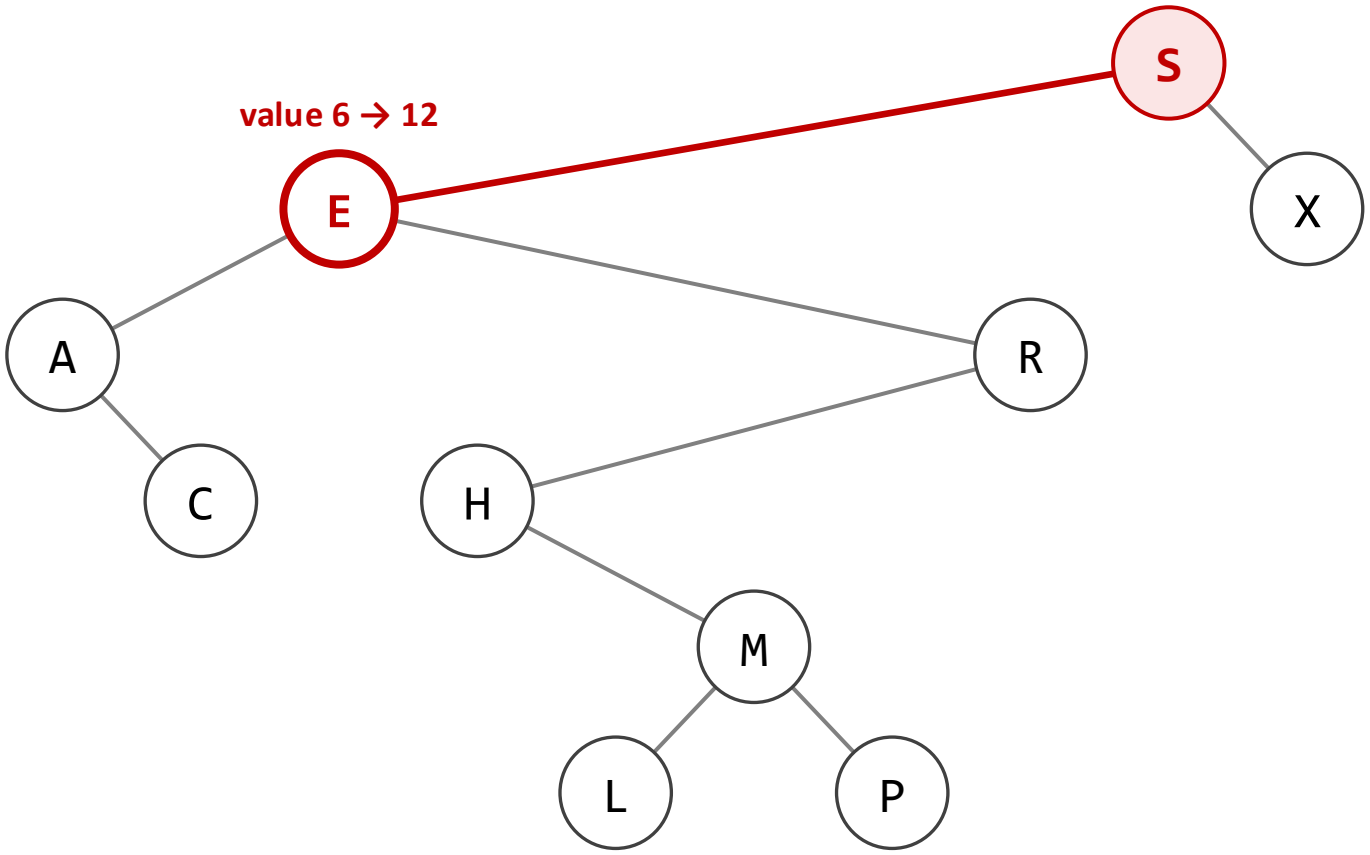
compares **18**

+14: M 4, P 5, L 5

new keys

the path they share

Building the tree: `put(E, 12)`



E one more time: 6 becomes 12.

13 puts, 10 keys: the same table as last video, now as a tree.

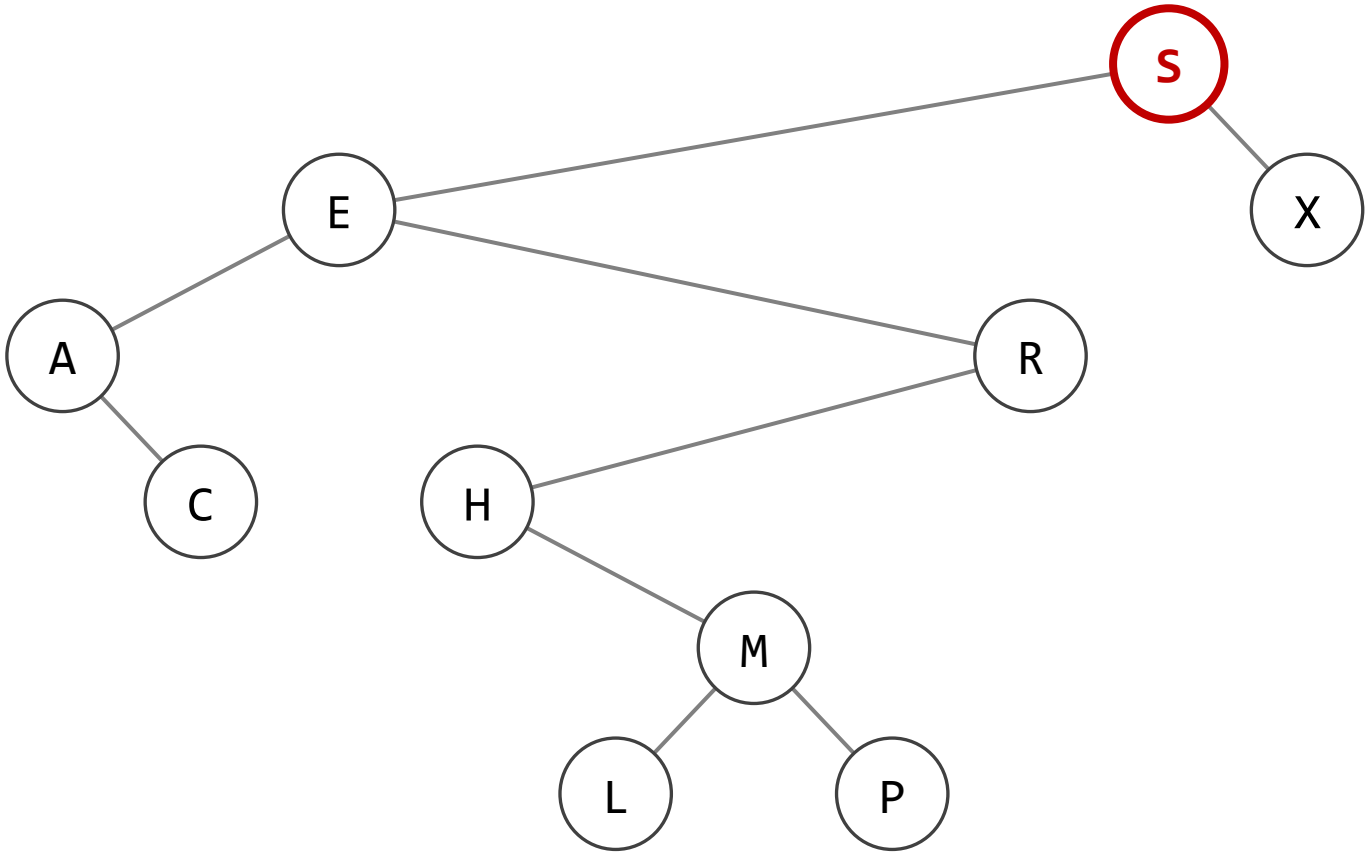
This is the tree we'll trace for the rest of this video.

Keys are drawn left to right in sorted order. That's not a coincidence (more later).

value replaced

the path

Search



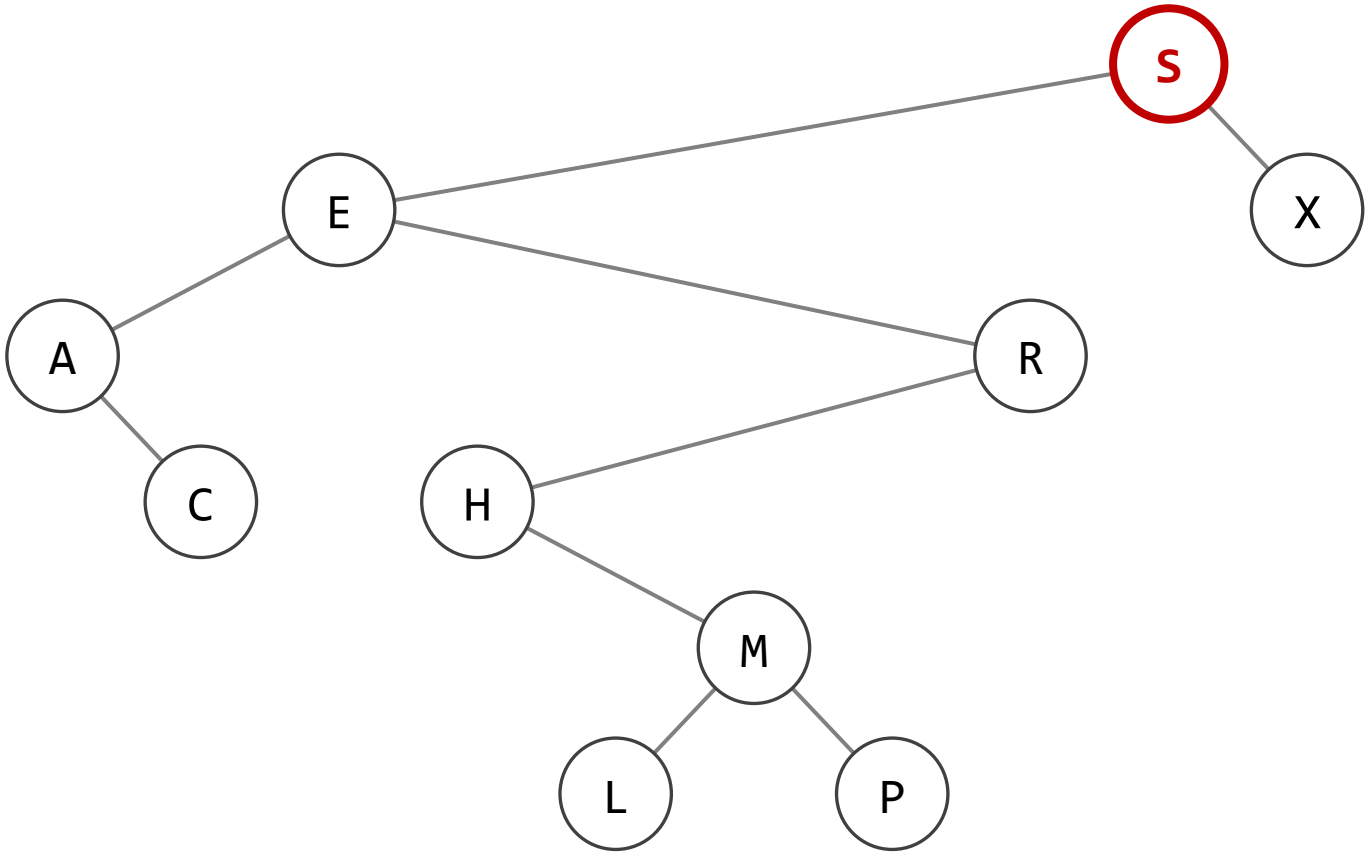
Start at the root. Compare:

- **less**: go left
- **greater**: go right
- **equal**: found it, a **search hit**

Fall off the tree onto a `null` link? It's not there: a **search miss**.

It's binary search, but the "middle" is wherever the tree says it is.

get(H)



H is **less** than S. Go **left**.

cost so far

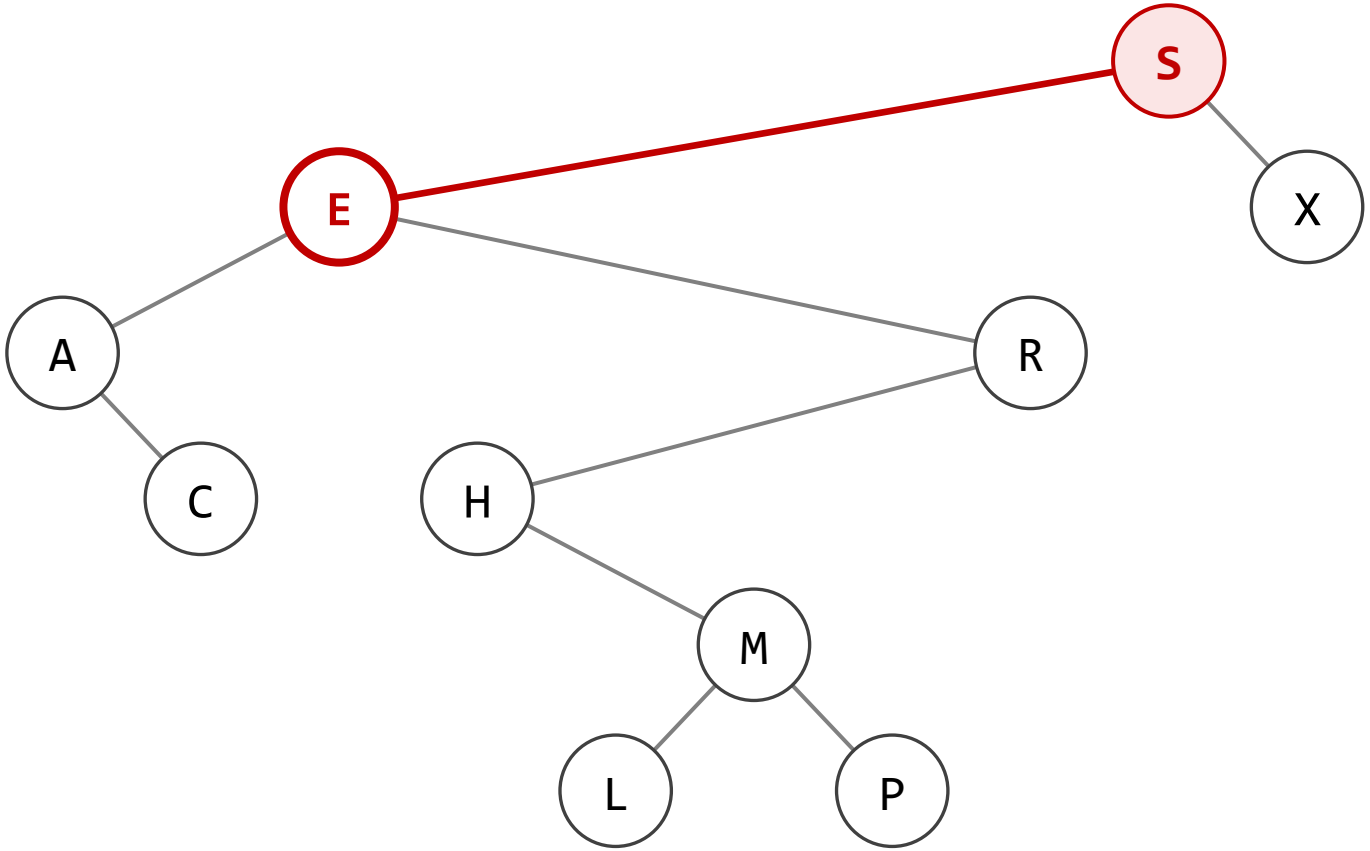
compares **1**

H vs. S

the node we're comparing with

the path so far

get(H)



H is **greater** than E. Go **right**.

cost so far

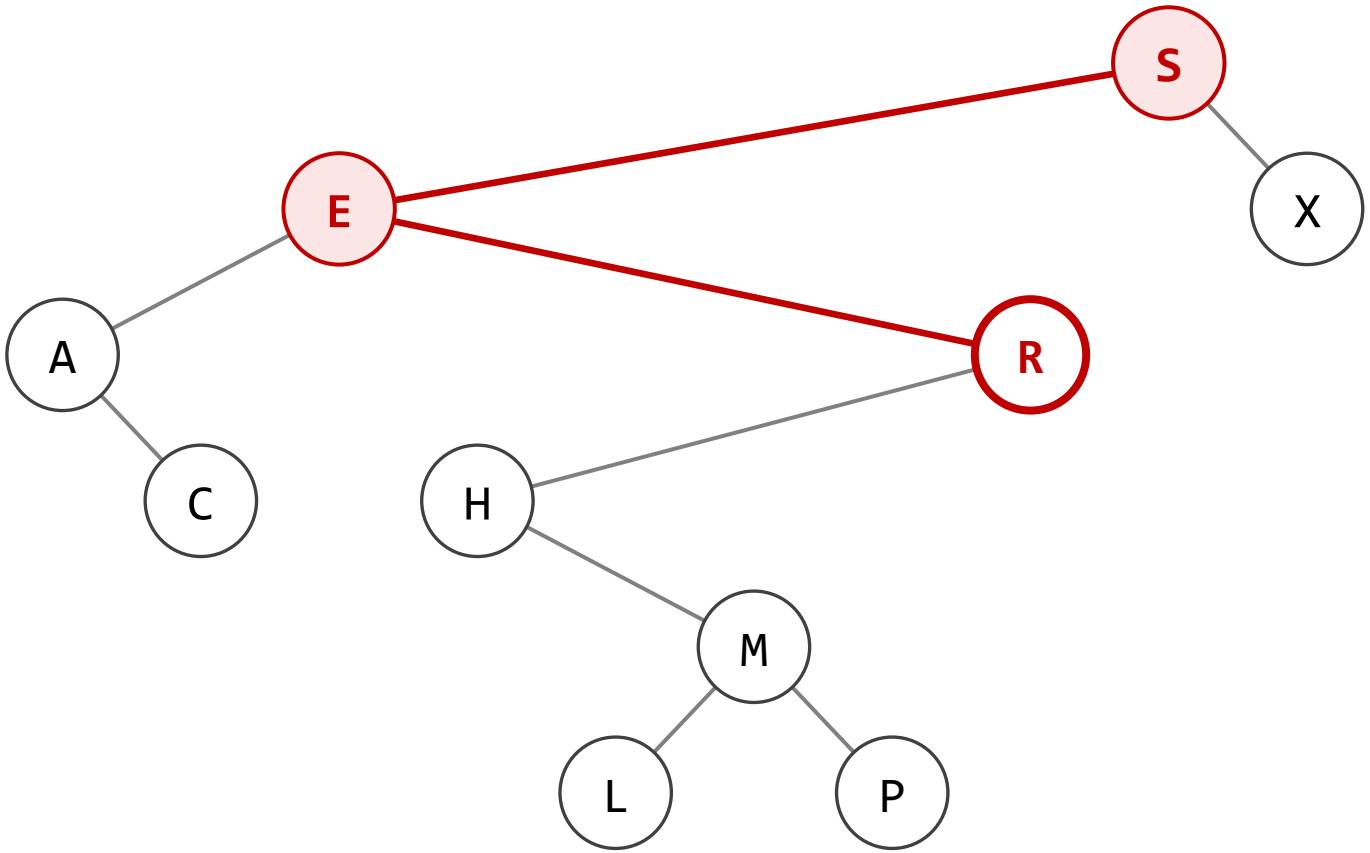
compares **2**

H vs. E

the node we're comparing with

the path so far

get(H)



H is **less** than R. Go **left**.

cost so far

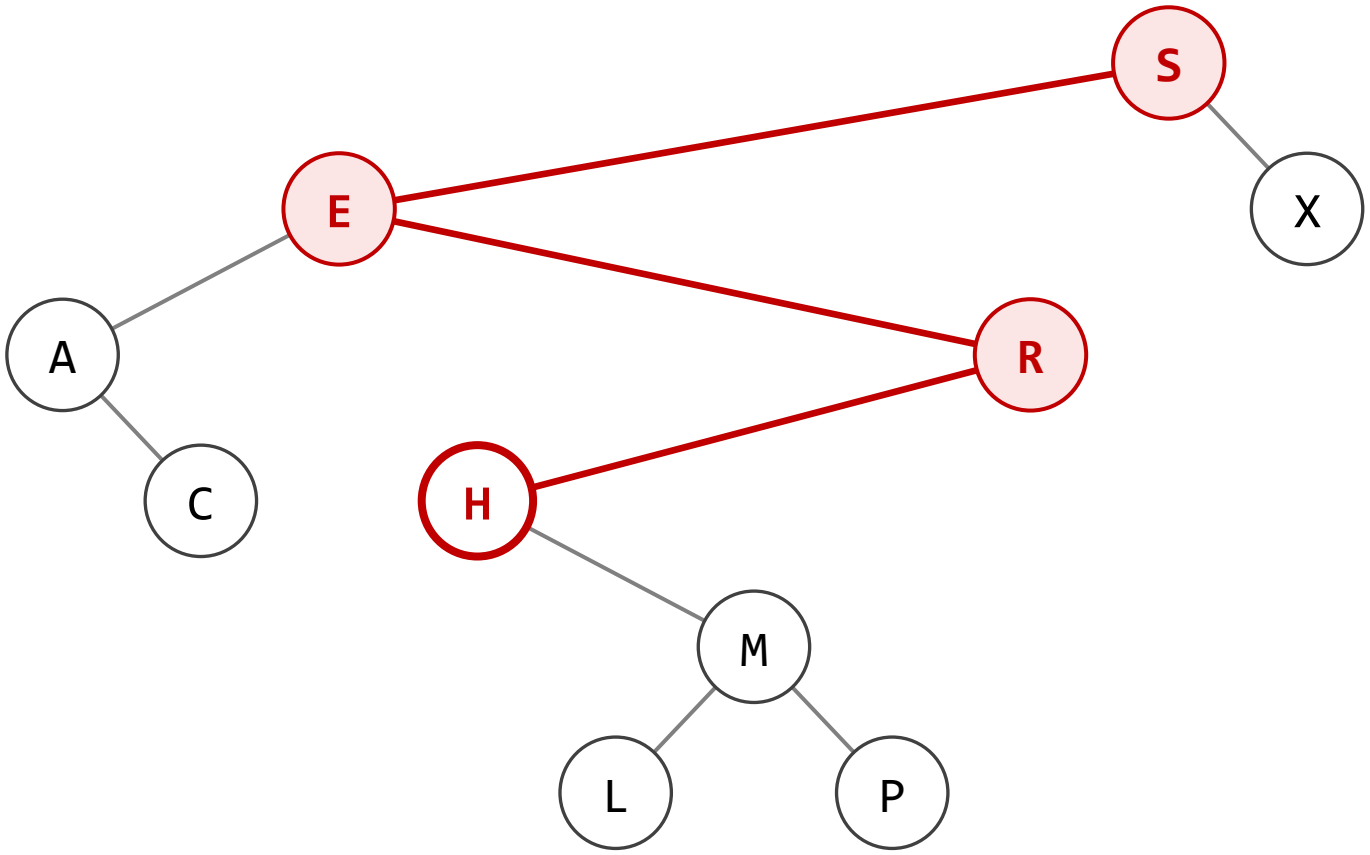
compares **3**

H vs. R

the node we're comparing with

the path so far

get(H)



H **equals** H. **Search hit!** Return its value, **5**.

4 compares: one per level. Cost of a search = **1 + depth** of the node.

A million keys inserted in random order: about **28** compares, on average.

cost so far

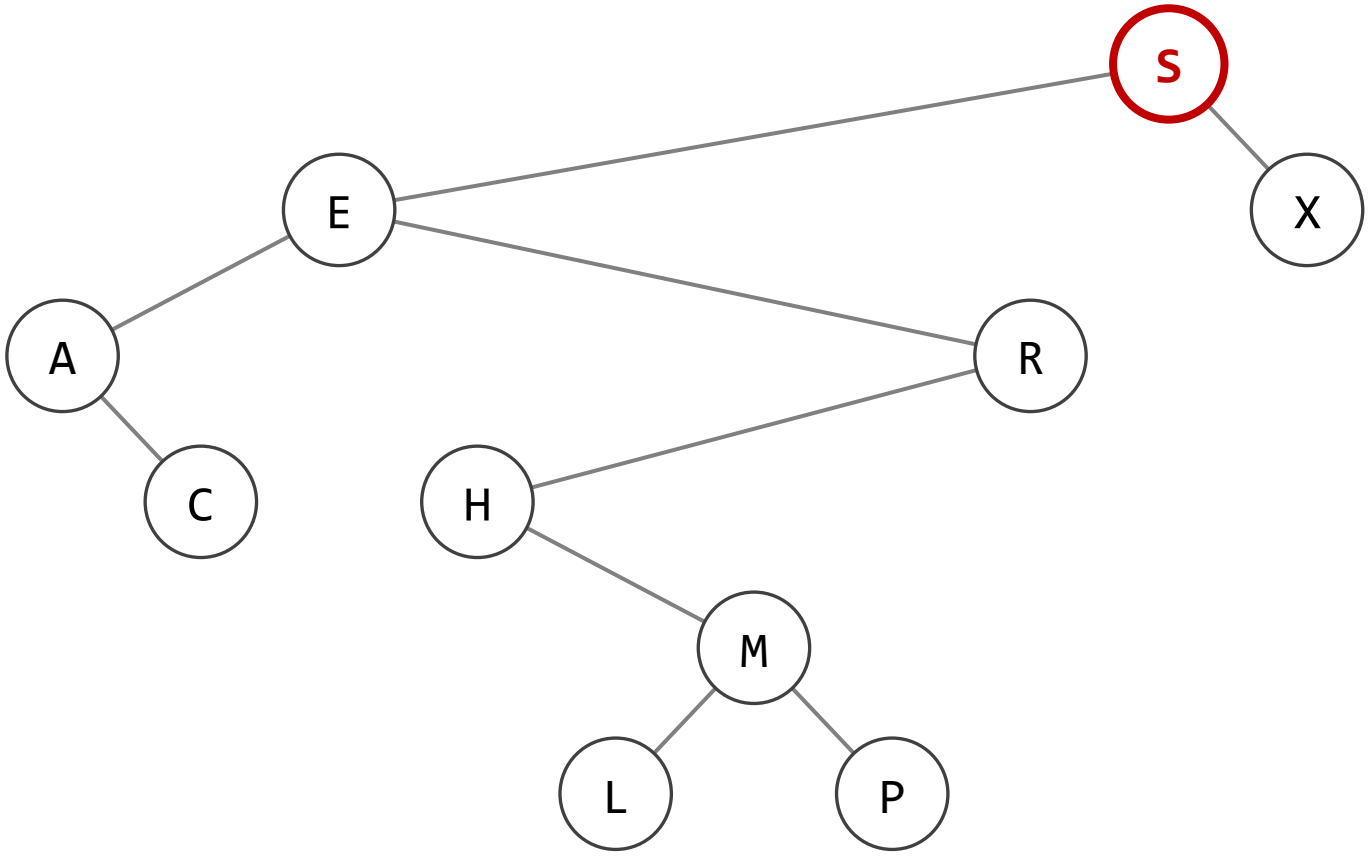
compares **4**

H vs. H: found it

the node we're comparing with

the path so far

get(G)



G is **less** than S. Go **left**.

G and H are neighbors in sorted order, so G's search follows H's path...

cost so far

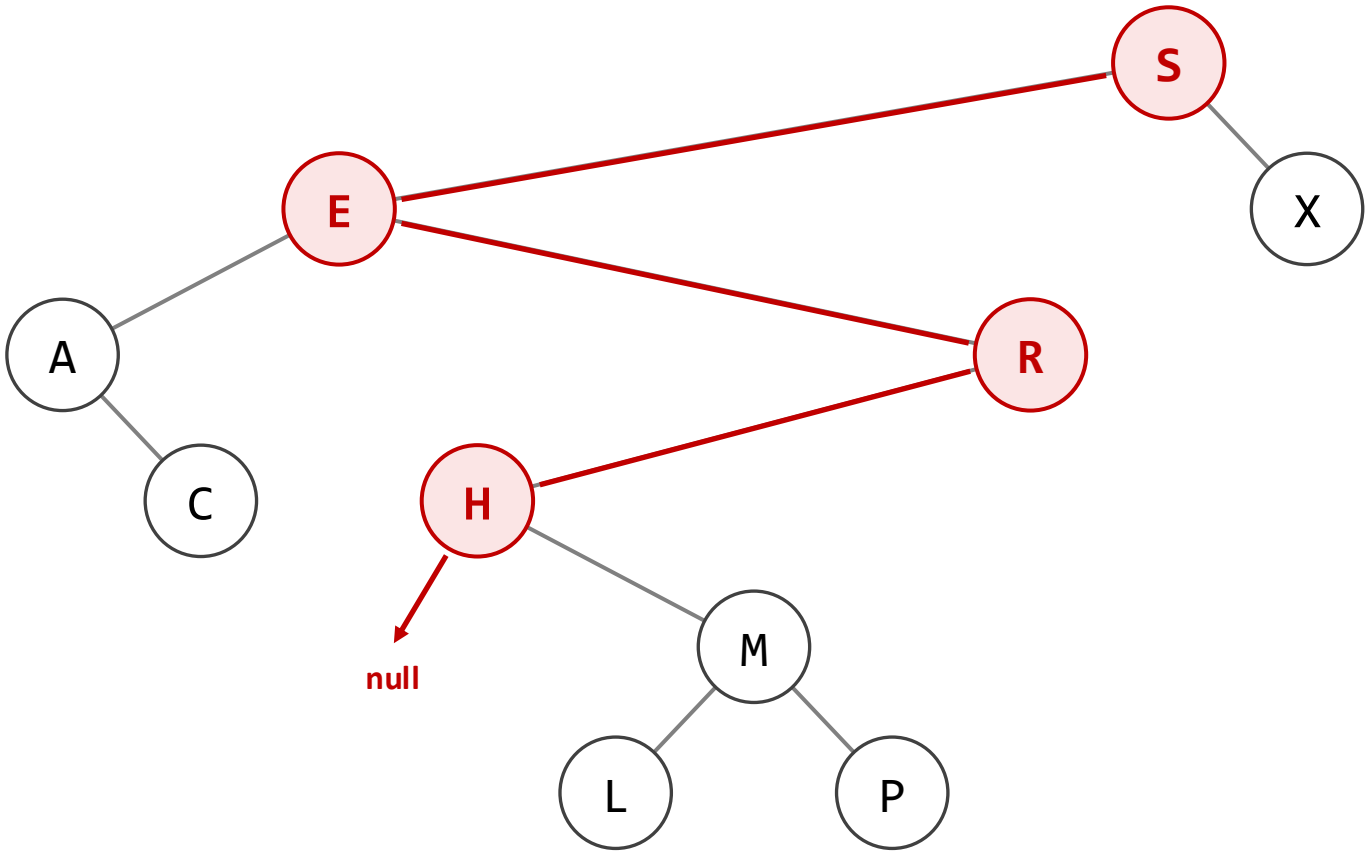
compares **1**

G vs. S

the node we're comparing with

the path so far

get(G)



Right at E, left at R, left at H, and H's left link is `null`. **Search miss**: return `null`.

That `null` link is exactly **where G would go**.

A miss costs 1 compare per node on the path. A million random keys: about **28** again.

cost so far

compares **4**

+3: E, R, H

the node we're comparing with

the path so far

`get`: the code

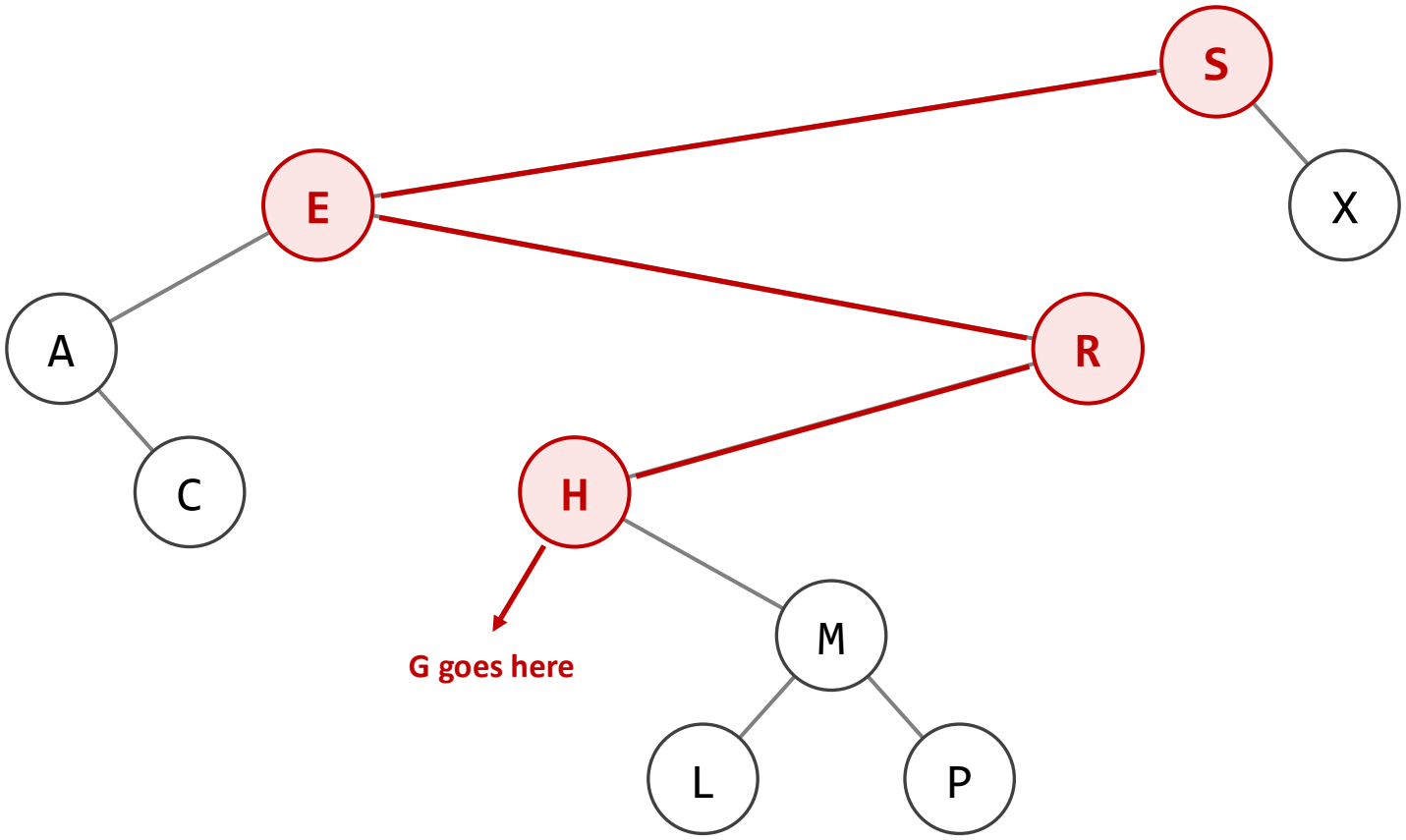
```
1  public Value get(Key key) {
2      Node x = root;
3      while (x != null) {
4          int cmp = key.compareTo(x.key);
5          if      (cmp < 0) x = x.left;
6          else if (cmp > 0) x = x.right;
7          else return x.val;          // hit
8      }
9      return null;                    // miss
10 }
```

A loop that walks one path from the root down.

Compare it with `rank()` in the ordered array: **same three cases**. `x = x.left` plays the part of `hi = mid - 1`.

Each trip around the loop is **one compare**. Cost: **1 + depth** of where the search ends.

put(G, 13)



Insert starts out as a **search**.

If we find G, replace its value. Done.

We don't: it's the same search as `get(G)`, S, E, R, H, ending at H's empty left link.

cost so far

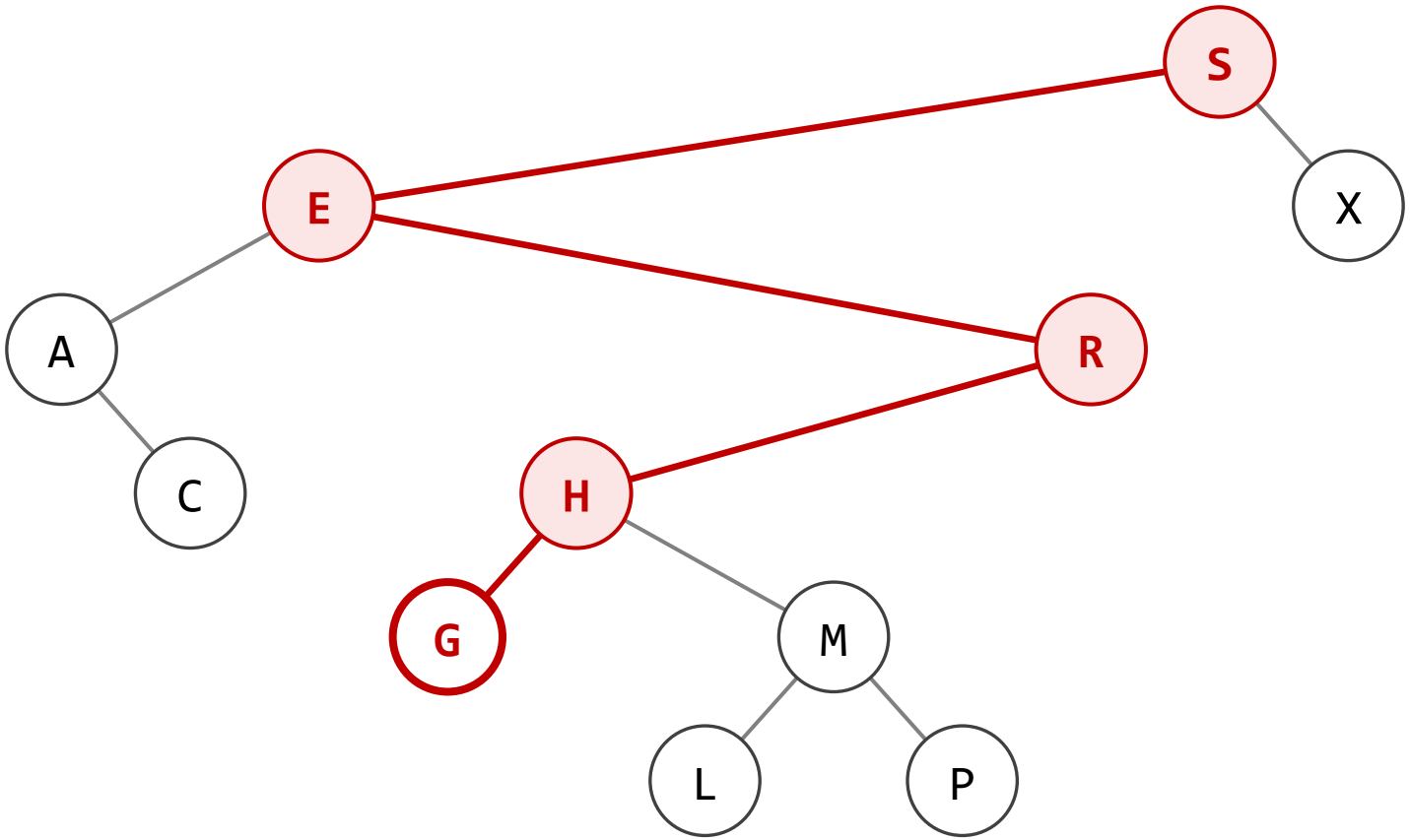
compares	4
moves	0

+4: S, E, R, H

the node we're comparing with

the path so far

put(G, 13)



Put a **new node** there. One link changes: H's left. **Nothing moves.**

Cost: **1 + depth**, the same as a search.

A million random keys: about **28** compares and **0** moves. The ordered array: about **500,000** moves.

cost so far

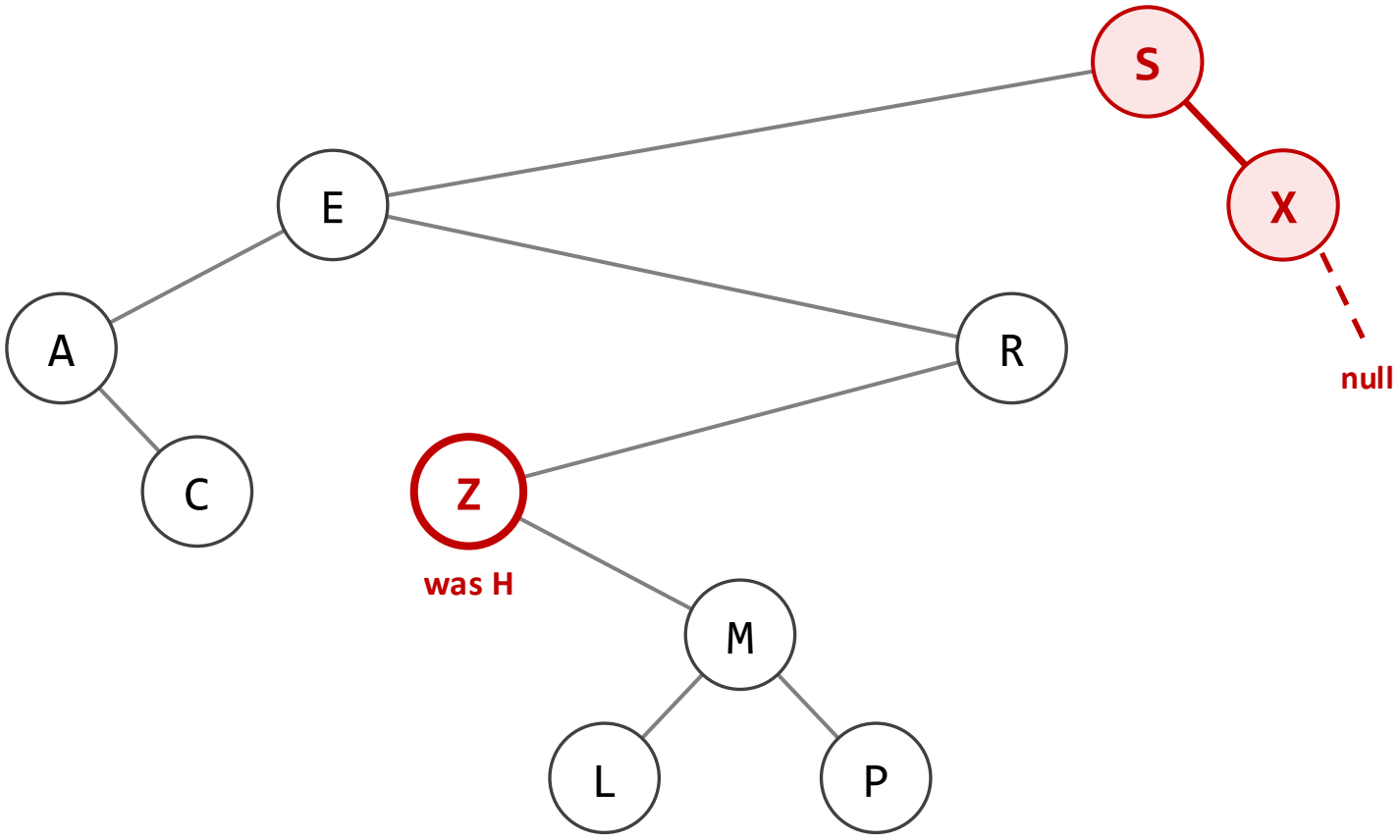
compares	4
moves	0

adding G: no compares

the new node

the search path

Keys must not change



Where a key goes is decided **once**, at `put` time, by compares.

Suppose the client later **changes** H's key to Z. Z sits on S's **left**, but Z > S.

`get(Z)`: right at S, right at X, `null`. **Search miss**, with Z right there in the tree.

So keys must never change once they're in the tree: use **immutable** keys, like `String` and `Integer`.

a key that changed

where get(Z) looks

put: the code

```
1  public void put(Key key, Value val) {
2      root = put(root, key, val);
3  }
4
5  private Node put(Node x, Key key, Value val) {
6      if (x == null) return new Node(key, val);
7      int cmp = key.compareTo(x.key);
8      if      (cmp < 0) x.left  = put(x.left,  key, val);
9      else if (cmp > 0) x.right = put(x.right, key, val);
10     else              x.val   = val;
11     return x;
12 }
```

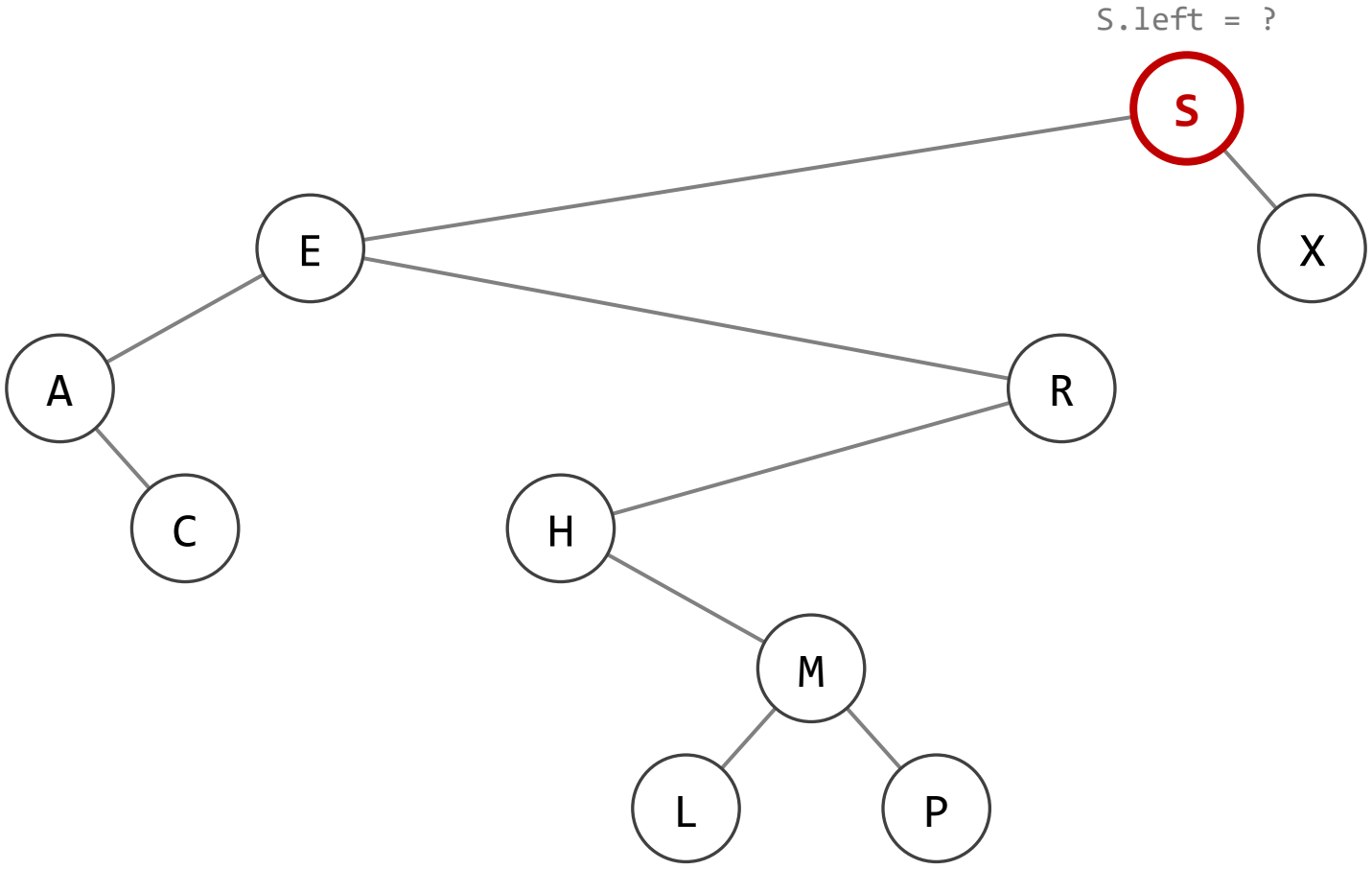
Recursive: "put this key into the tree rooted at x, and **give me back** the root of the result."

Line 6: the empty tree. Make a node.

Lines 8–9: put it in the correct subtree, and **reset the link** to whatever comes back.

Short, but tricky. Next slide: what actually happens.

Inside `put(G, 13)`: going down



the calls

```
root = put(S, G)
 S.left = put(E, G)
```

G < S, so `put(S, G)` calls put on S's left subtree.

S **waits** for the answer, to store it in `S.left`.

cost so far

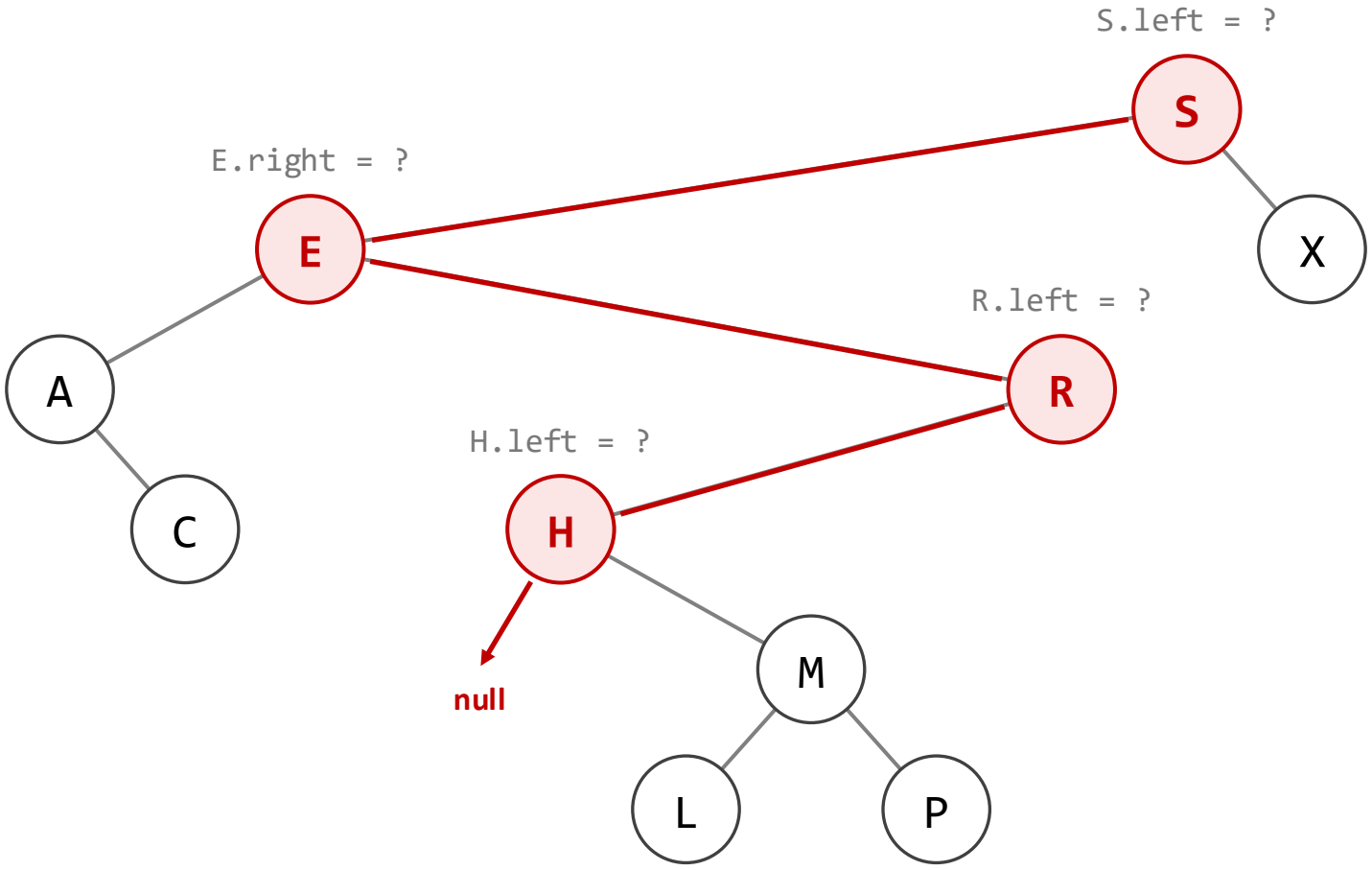
links set	0
changed	0

going down: nothing set yet

the call that's running

calls waiting for an answer

Inside `put(G, 13)`: going down



the calls

```
root = put(S, G)
 S.left = put(E, G)
  E.right = put(R, G)
   R.left = put(H, G)
    H.left = put(null, G)
     return new Node(G)
```

E, R and H do the same: each **waits** with a half-finished assignment.

H's left is `null`: the base case makes the new node.

cost so far

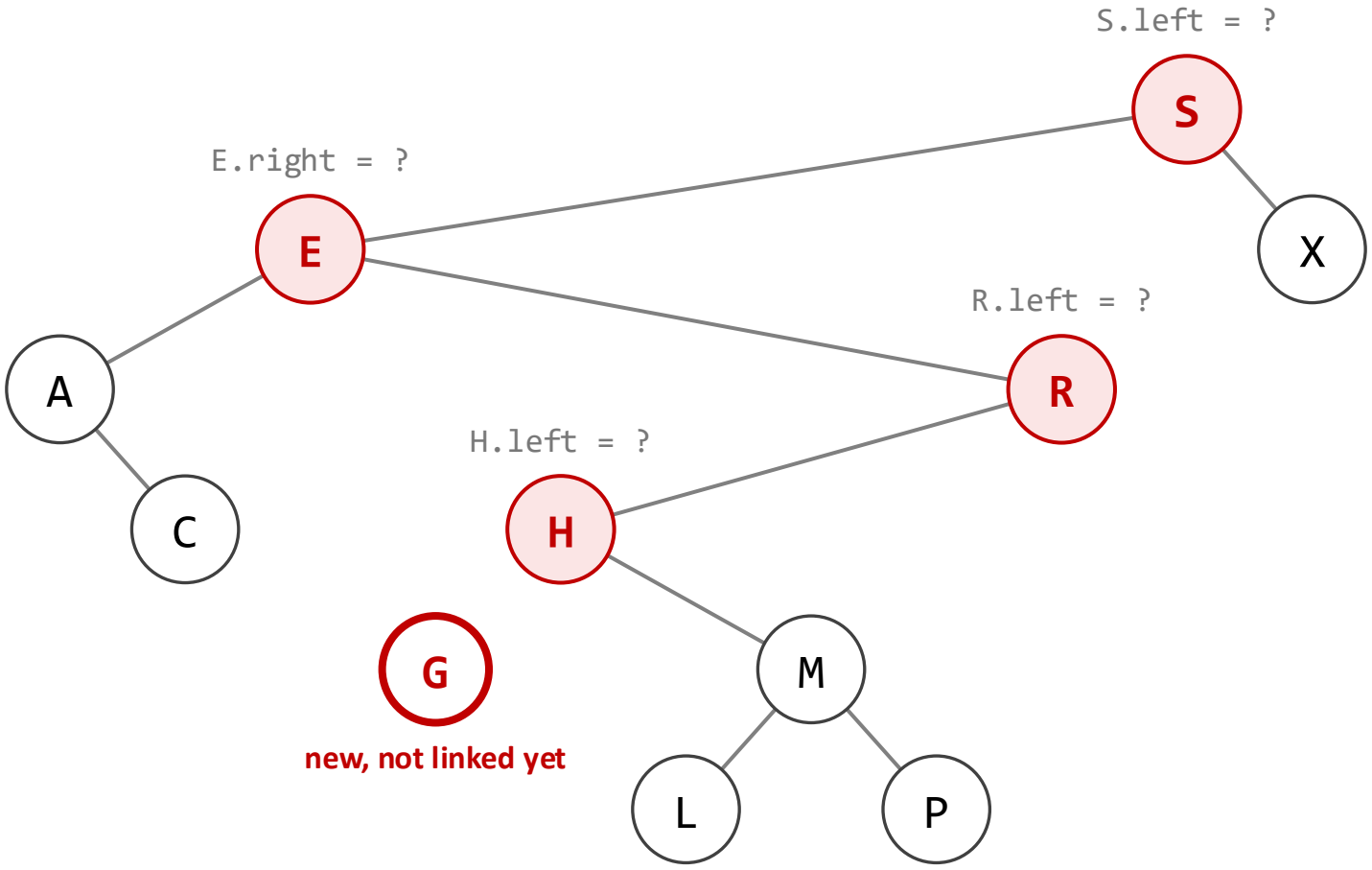
links set	0
changed	0

still nothing set

the call that's running

calls waiting for an answer

Inside `put(G, 13)`: coming back up



the calls

```
root = put(S, G)
 S.left = put(E, G)
  E.right = put(R, G)
   R.left = put(H, G)
    H.left = put(null, G)
     return new Node(G)
```

`put(null, G)` returns the new node **to H's call**, which was waiting for it.

Nothing points to G yet.

cost so far

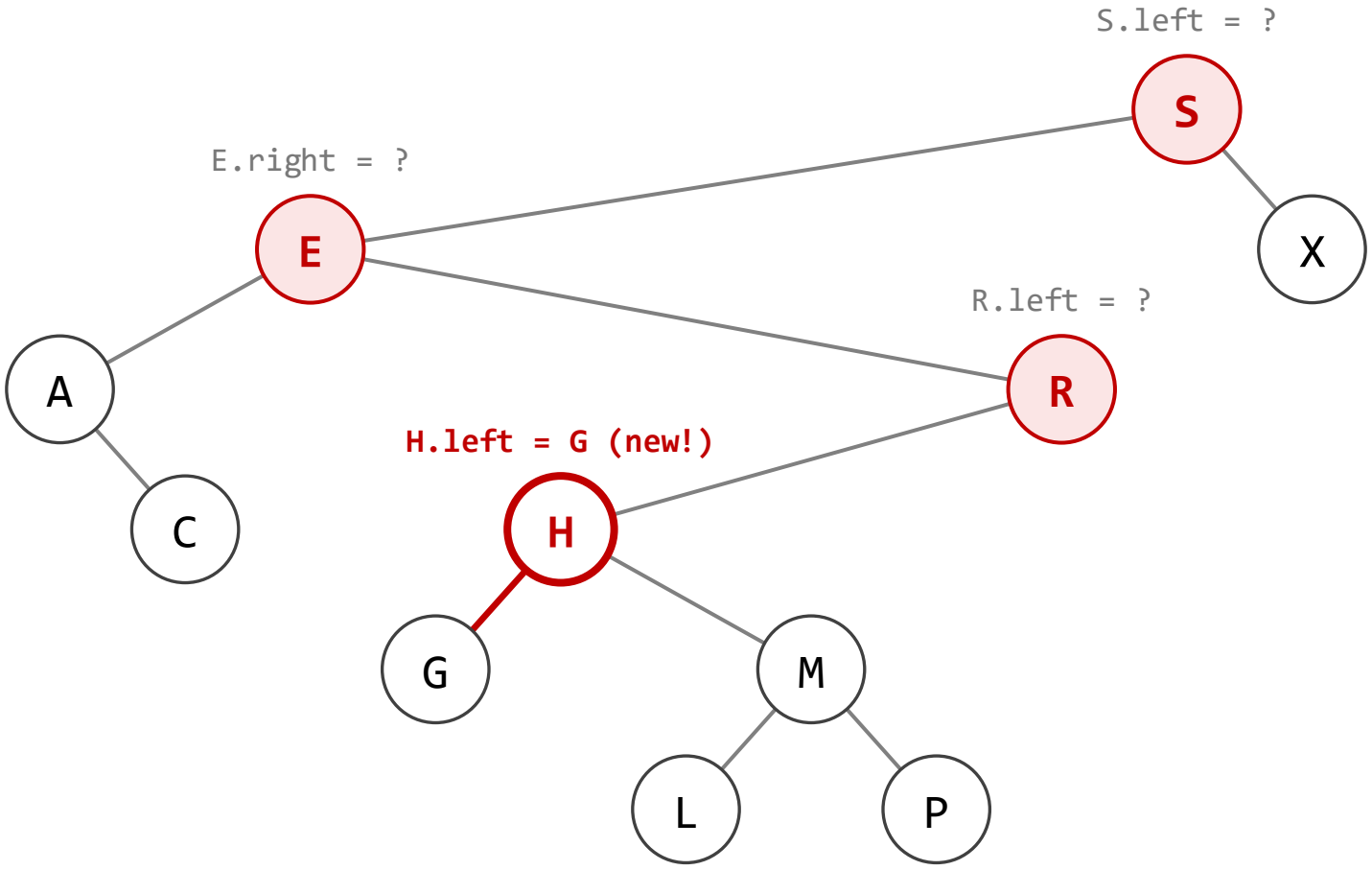
links set	0
changed	0

returning: no link set

the call that's running

calls waiting for an answer

Inside `put(G, 13)`: coming back up



the calls

```
root = put(S, G)
 S.left = put(E, G)
  E.right = put(R, G)
   R.left = put(H, G)
    H.left = put(null, G)
     return new Node(G)
```

H stores the answer: `H.left = G`, the **only** link that really changes.

Then H returns **H itself** to R's call.

cost so far

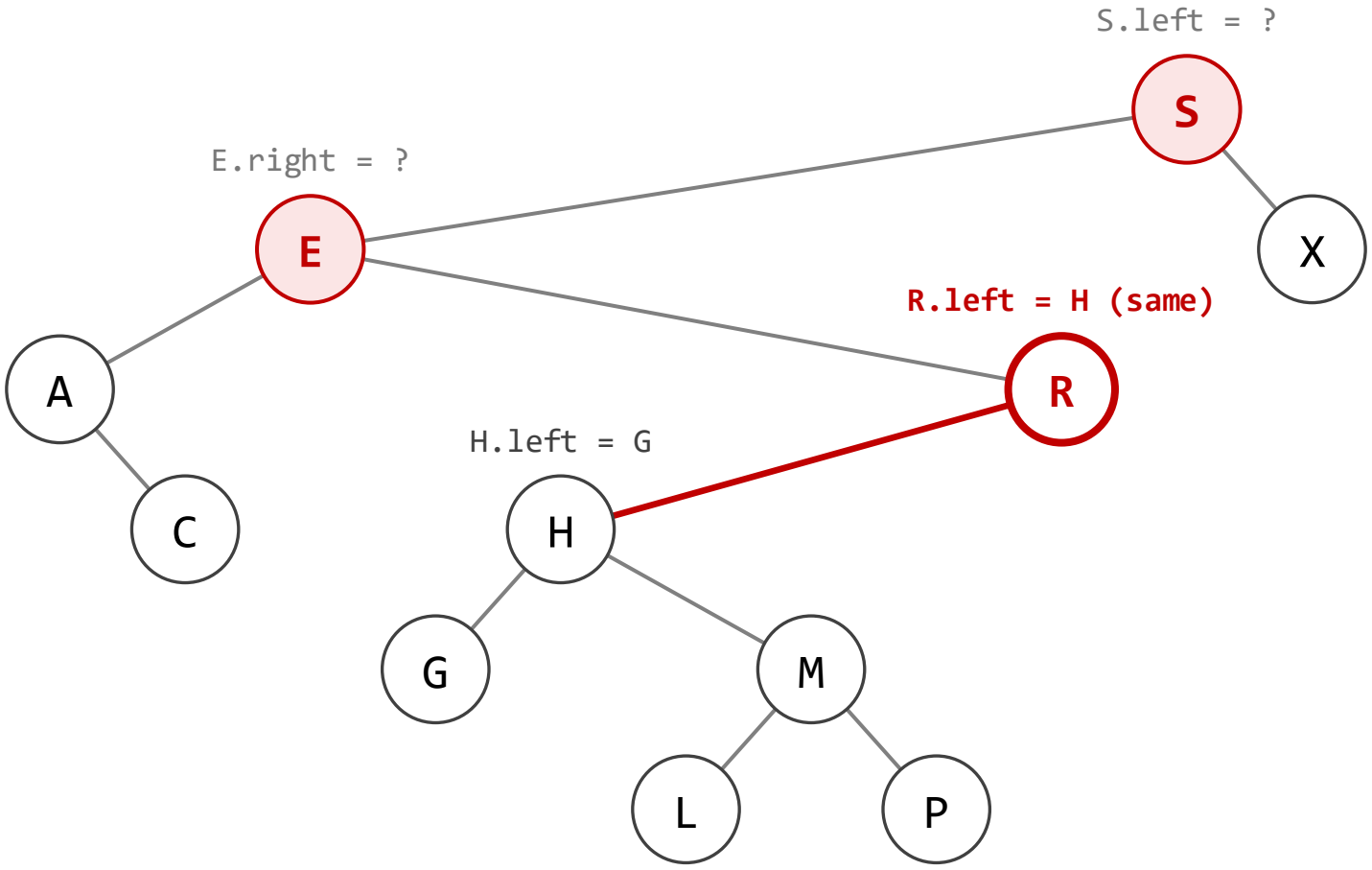
links set	1
changed	1

H.left = G

the call that's running

calls waiting for an answer

Inside `put(G, 13)`: coming back up



the calls

```
root = put(S, G)
 S.left = put(E, G)
  E.right = put(R, G)
   R.left = put(H, G)
    H.left = put(null, G)
     return new Node(G)
```

R gets H back and stores it: `R.left = H`.

It was already H. **Nothing changes.** R returns R.

cost so far

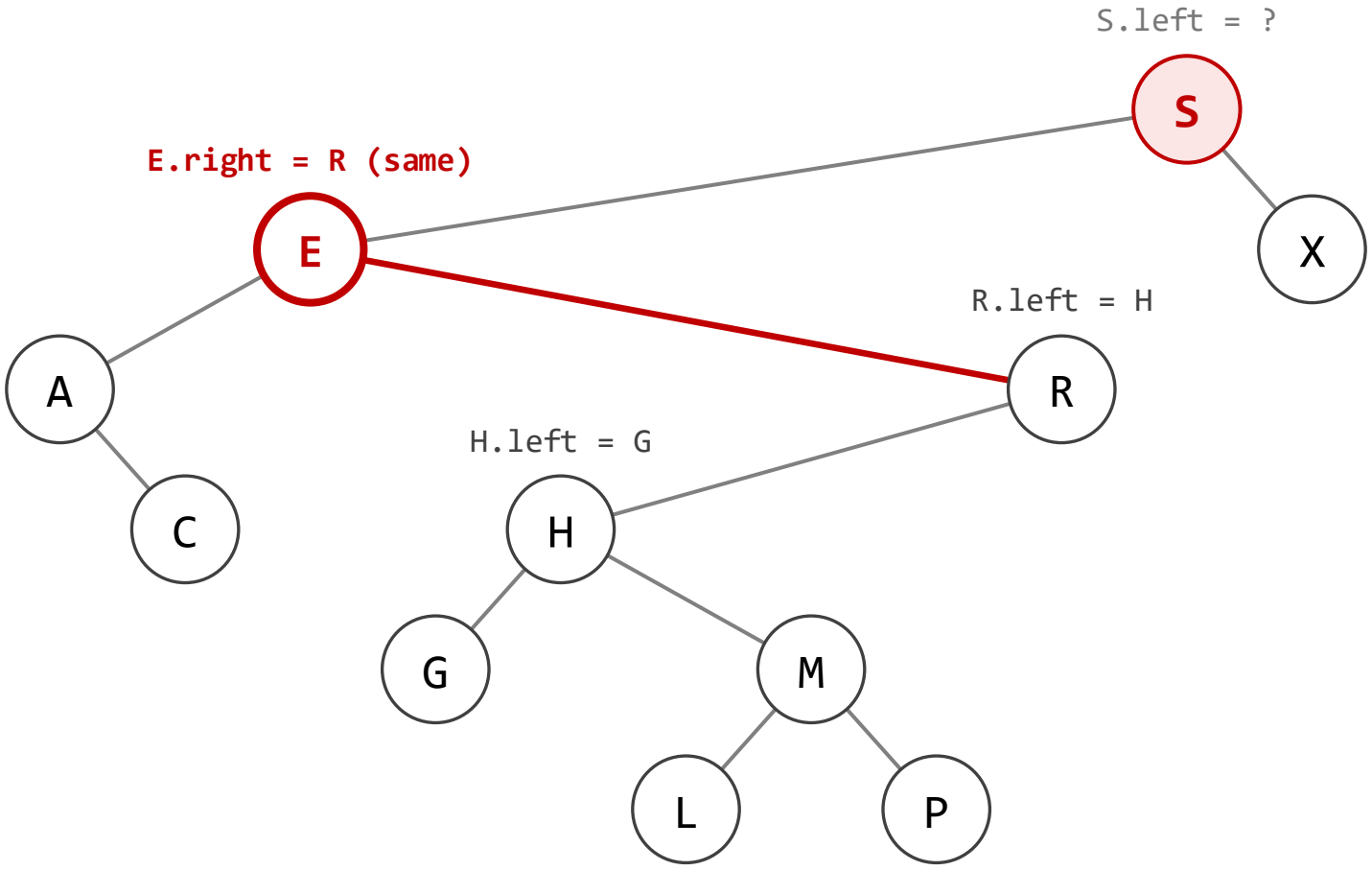
links set	2
changed	1

R.left = H: same link

the call that's running

calls waiting for an answer

Inside `put(G, 13)`: coming back up



the calls

```
root = put(S, G)
 S.left = put(E, G)
  E.right = put(R, G)
   R.left = put(H, G)
    H.left = put(null, G)
     return new Node(G)
```

E gets R back: `E.right = R`. Same as before.

E returns E.

cost so far

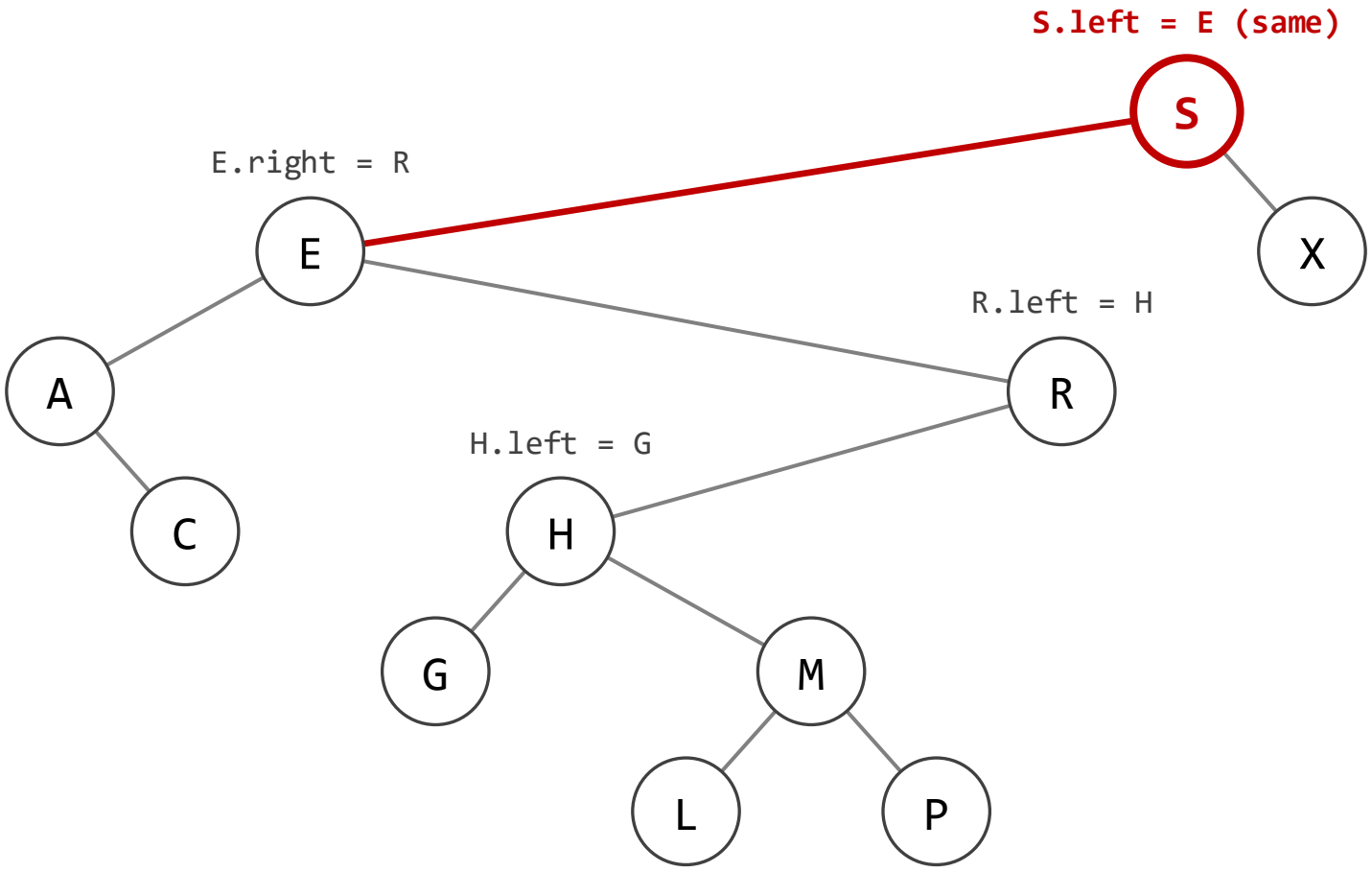
links set	3
changed	1

E.right = R: same link

the call that's running

calls waiting for an answer

Inside `put(G, 13)`: coming back up



the calls

```
root = put(S, G)
 S.left = put(E, G)
  E.right = put(R, G)
   R.left = put(H, G)
    H.left = put(null, G)
     return new Node(G)
```

S gets E back: `S.left = E`. Same as before.

S returns S, to the public put.

cost so far

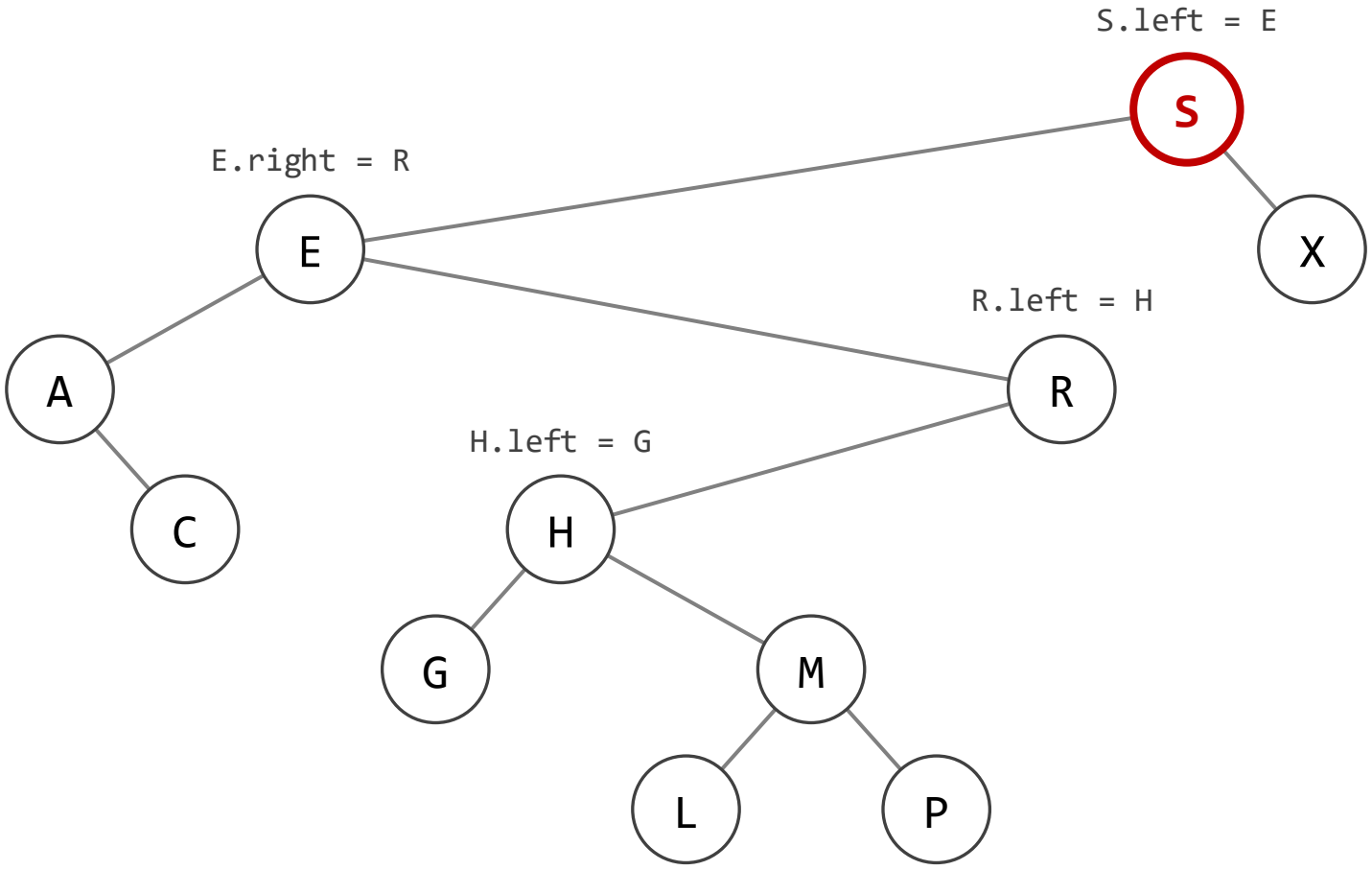
links set	**4**
changed	1

S.left = E: same link

the call that's running

calls waiting for an answer

Inside `put(G, 13)`: coming back up



the calls

```
root = put(S, G)
 S.left = put(E, G)
  E.right = put(R, G)
   R.left = put(H, G)
    H.left = put(null, G)
     return new Node(G)
```

`root = S`: same root. **5 links set, 1 changed**: one cheap assignment per level.

Why bother? Next video, **deletion** needs some of them to change.

cost so far

links set	**5**
changed	1

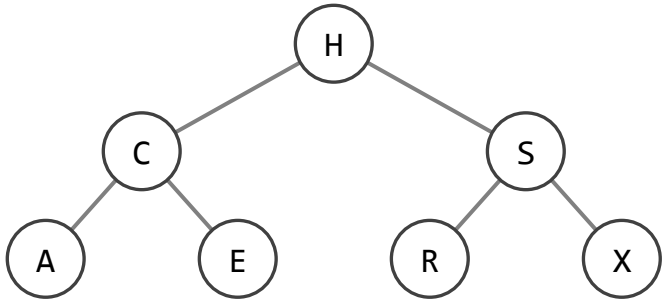
root = S: same link

the call that's running

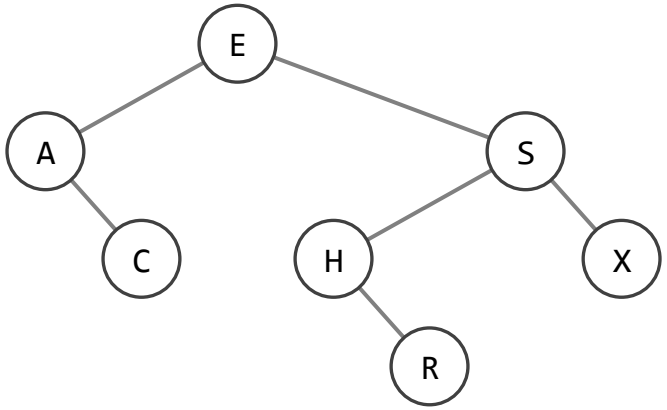
calls waiting for an answer

Same keys, different trees

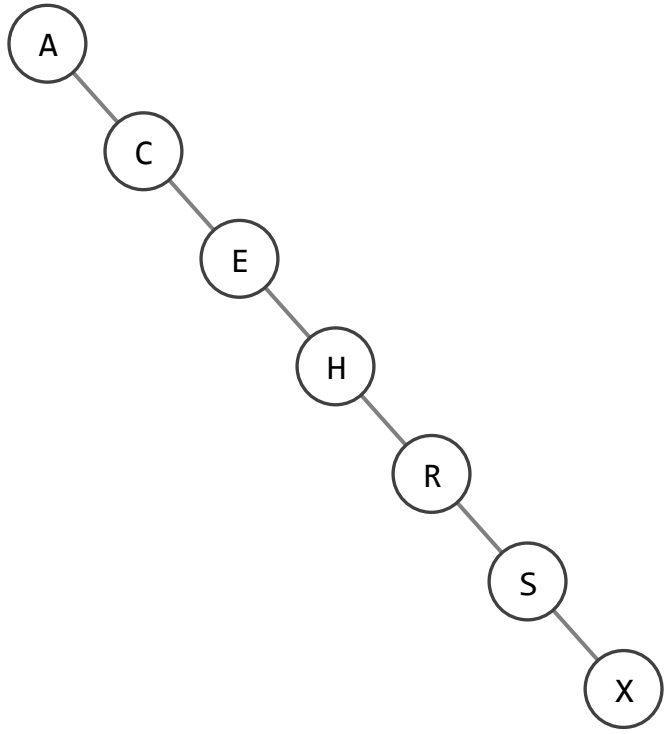
best case: H C S A E R X



typical: E S A X C H R

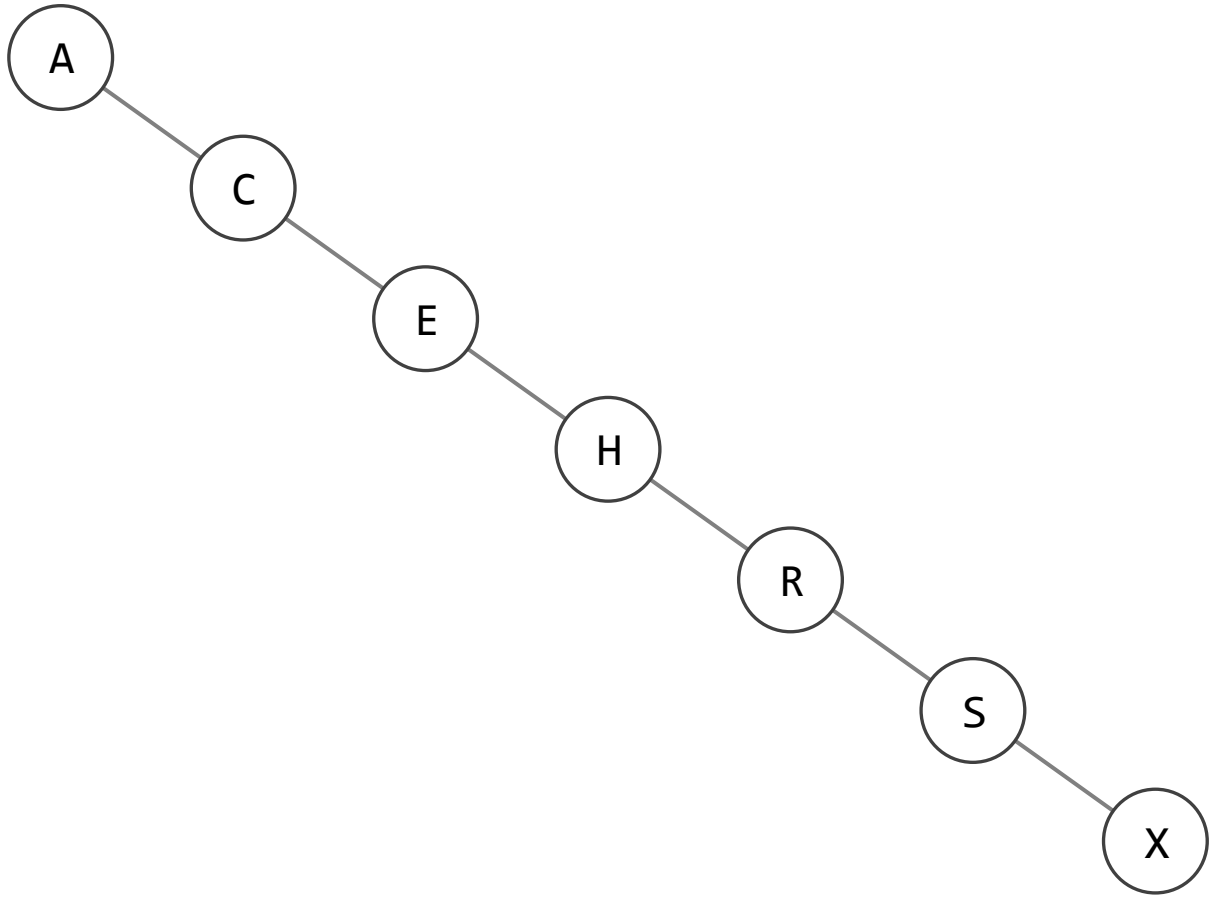


worst case: A C E H R S X



The same 7 keys. The **shape** depends only on the **order they were inserted** in. Cost of `get` and `put` = 1 + depth.

The worst case



Keys arrive **in sorted order**: every key goes to the right of the last one.

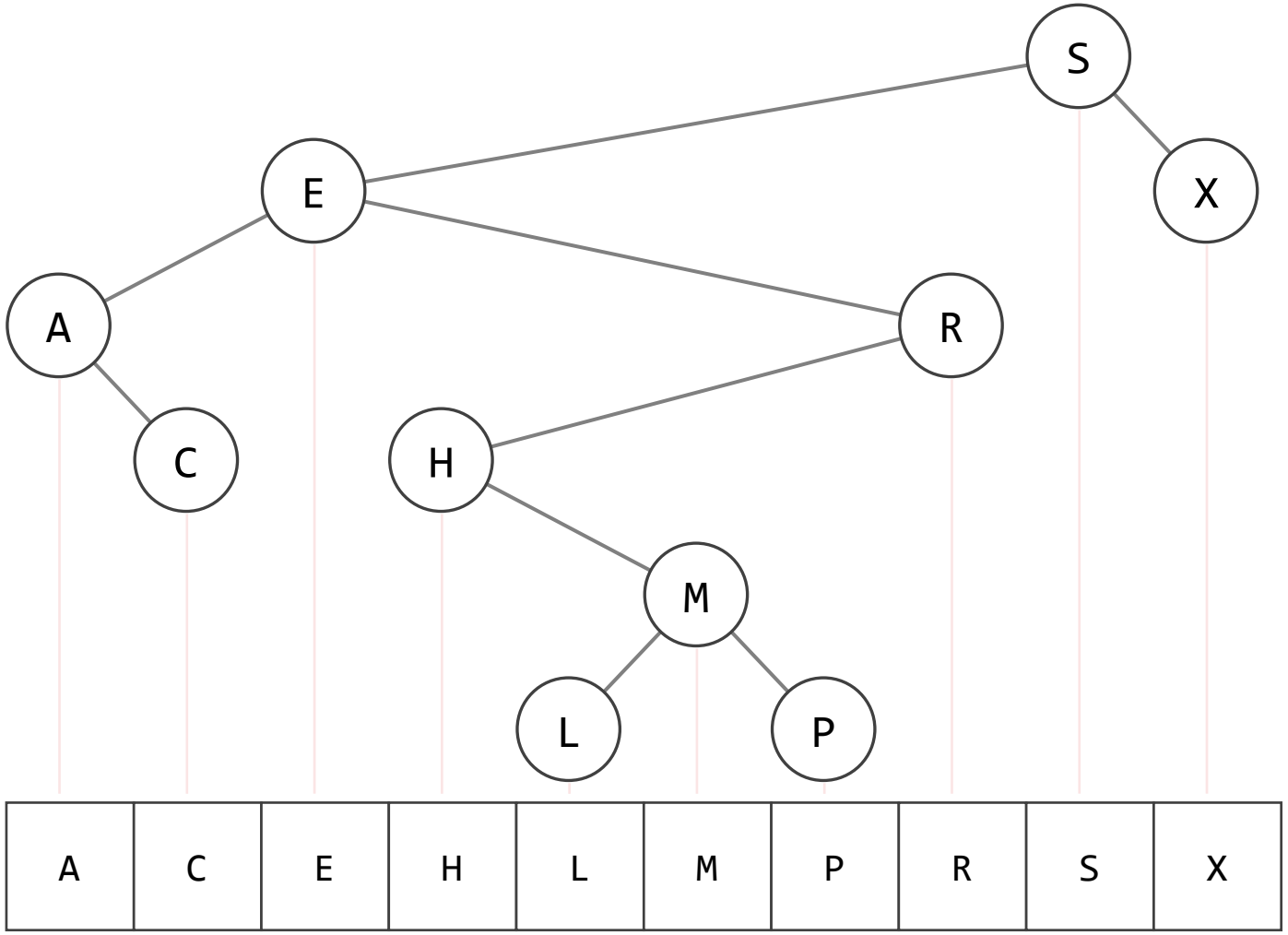
The "tree" is a linked list. Search and insert: **n** compares.

A million keys in sorted order: up to a **million** compares per search.

Sound familiar? It's **quicksort on sorted input** without the shuffle.

And sorted input is common in real life: timestamps, IDs, alphabetized files...

Bonus: a BST can sort



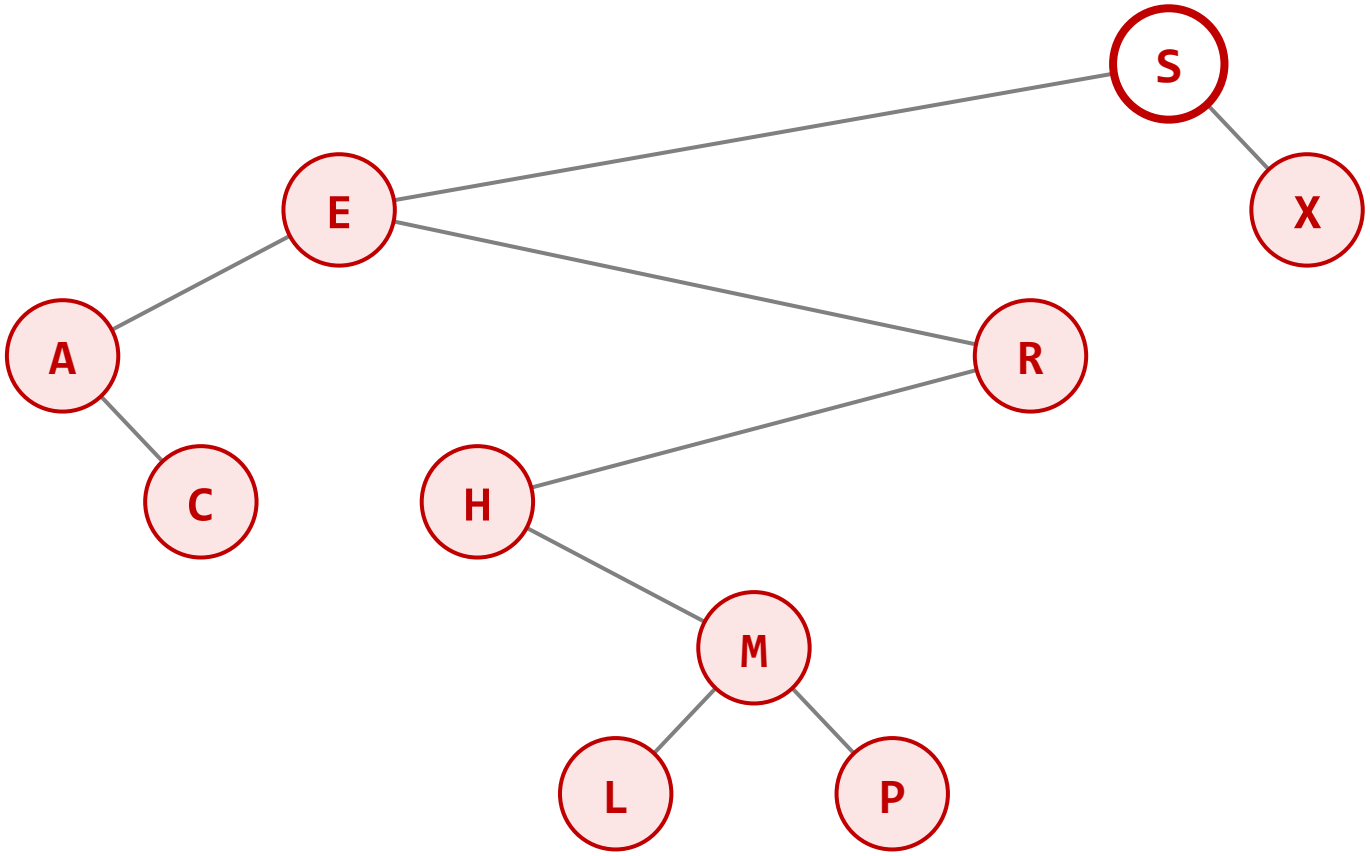
Read the tree **left to right** and the keys come out **in order**.

That's symmetric order at work: everything to a node's left is smaller.

So: shuffle the keys, insert them all, then read them off left to right. That's a sort.

How to "read left to right" in code: an in-order traversal, next video.

BSTs and quicksort



The first key in, S, is the root. Every later key is compared with S and sent **left** (smaller) or **right** (bigger).

That's **partitioning**, with S as the partitioning item.

Then E partitions the left side, and so on, all the way down.

A BST makes **the same compares** as quicksort, if quicksort used the first key as its partitioning item.

S, the "partitioning item"

everything it got compared with

How fast is a BST, on average?

If the n keys arrive in **random order**, the BST is built like quicksort on a shuffled array. So we get to borrow quicksort's math:

random insertion order	**n = 1,000**	**n = 1,000,000**
average compares for a search or insert: ~2 ln n ≈ **1.39 lg n**	~14	~28
average height of the tree: ~4.31 ln n	~30	~60
worst case (sorted input): n	**1,000**	**1,000,000**

Random order makes the worst case **astronomically unlikely**. But we don't get to pick the order keys arrive in.

(The height result is from 2003. People were still working out the details of this 1960s data structure 40 years later.)

The scoreboard so far

implementation	search	insert	avg. search hit	avg. insert
sequential search	n	n	n / 2	n
binary search	lg n	n	lg n	n / 2
BST	**n**	**n**	**1.39 lg n**	**1.39 lg n**

On average, the BST finally gets **both** operations down to logarithmic.

The worst case is still n, and it happens for **sorted input**.

Question to think about: quicksort fixed its worst case by shuffling. Why can't a symbol table just shuffle its keys?

Why not just shuffle?

Quicksort gets the **whole array** before it starts, so it can shuffle first.

A symbol table gets keys **one** `put` **at a time**, whenever the client feels like it. It can't shuffle keys it hasn't seen yet.

So a BST's shape is up to the client. A client with sorted data gets the worst case.

The real fix: a tree that **keeps itself balanced** no matter what order keys arrive in. That's §3.3, **balanced search trees**.

Coming up: the rest of the BST

Next video: everything **else** a symbol table can do, on a BST.

- `min`, `max`, **`floor`**, `ceiling`
- **`rank`** and `select`, with one extra field per node
- reading the keys in order
- **deletion**, which turns out to be the hard part